

Akaverse: Boosting Sharded Blockchain via Multi-Leader Parallel Pipelines

Xinlei Sheng, Jian Shen, *Member, IEEE*, Lei Xie, Haoyu Ma, *Member, IEEE*, Chen Wang, *Member, IEEE*, Q. M. Jonathan Wu, *Senior Member, IEEE*

Abstract—Sharded blockchains offer horizontal scalability. However, existing per-shard consensus protocols serialize execution through a single leader and transaction pool, creating a bottleneck that intensifies head-of-line blocking, increases coordination latency under skewed workloads and in public network environments, and ultimately prevents the realization of linear scalability. In addition, the single-leader paradigm centralizes the failure surface, thereby undermining the liveness and safety of sharded blockchain systems. To address these limitations, we propose Akaverse, an intra-shard protocol based on multi-leader pipelines. Unlike existing approaches, Akaverse handles cross-shard coordination by mapping it to parallel execution across multiple transaction pools within each shard. Each pool runs an independent threshold-BLS consensus with constant-size certificates and view-synchronization gating. In addition, a global buffer pool (GBP) batches cross-pool interactions into fixed windows, providing deterministic ordering for certified records and rollback free step-level cross-pool coordination. Evaluation and analysis show that Akaverse preserves safety and liveness under partial synchrony and is resilient to forking attacks while maintaining protocol consistency. At the same time, Akaverse significantly and consistently outperforms existing blockchain sharding schemes in public network settings in terms of both throughput and transaction latency.

Index Terms—Dependable distributed systems, sharded blockchain, parallel pipelines, secure cross-shard coordination, adversarial resilience.

I. INTRODUCTION

BLOCKCHAIN sharding technology originated with the pioneering work of Elastico [1]. Drawing inspiration from traditional database sharding [2], Elastico introduced the idea of partitioning consensus tasks into parallel subdomains.

Xinlei Sheng is with the School of Computing Science and Technology (School of Artificial Intelligence), Zhejiang Sci-Tech University, Hangzhou 310018, China, and also with City University of Macau, Taipa, Macau, China. E-mail: xinlei_sheng@126.com.

Jian Shen, Lei Xie, and Chen Wang are with the School of Information Science and Engineering (School of Cyber Science and Technology), the Zhejiang Key Laboratory of Digital Fashion and Data Governance, and the Zhejiang Provincial International Cooperation Base for Science and Technology on Cloud Computing Security and Data Aggregation, Zhejiang Sci-Tech University, Hangzhou 310018, China. They are also with the Zhejiang Key Laboratory of Artificial Intelligence of Things (AIoT) Network and Data Security, Hangzhou 310053, China. They are also with the Zhejiang Provincial Innovation Center of Advanced Textile Technology, Shaoxing 312000, China. E-mail: s_shenjian@126.com; jhsx5679@gmail.com; wangchen@zstu.edu.cn.

Haoyu Ma is with the Beijing Jiaotong University, Beijing 100044, China. E-mail: hyma1@bjtu.edu.cn.

Q. M. Jonathan Wu is with the Department of Electrical and Computer Engineering, University of Windsor, Windsor, SL4 3BB, Ontario, Canada. E-mail: jwu@uwindsor.ca.

Corresponding author: Jian Shen (E-mail: s_shenjian@126.com).

Sharding addresses the scalability bottleneck inherent in the blockchain trilemma and has since attracted significant attention. Subsequent schemes such as OmniLedger [3] and RapidChain [4] introduced further improvements to key aspects of sharded blockchains, including cross-shard atomicity and dynamic committee management, thereby advancing their practical viability and robustness.

The core idea of sharding is to decompose a global blockchain network into multiple bounded-size consensus committees. Each committee, assigned through a randomized mechanism, processes a disjoint subset of transactions independently. By employing a parallel byzantine consensus model, such architectures theoretically enable near-linear scalability in system throughput. However, this horizontal scalability comes at the cost of both security and overhead. **Specifically, reducing the size of individual shard committees lowers the absolute number of Byzantine replicas required to compromise a shard, thereby increasing the risk of shard-level safety violations such as double-spending attacks.** Furthermore, ensuring the atomicity of cross-shard transactions requires multi-phase locking protocols, leading to a superlinear increase in confirmation latency as the proportion of cross-shard interactions grows [5].

These limitations become particularly pronounced when sharding scales to large deployments and operates under high cross-shard concurrency. In such settings, the system is increasingly constrained by these bottlenecks. (i) Certification and view-change handling often introduce substantial control-plane overhead, making it difficult to sustain stable progress under leader changes and network jitter. (ii) Decentralized randomness and reconfiguration mechanisms often introduce substantial integration and confirmation overhead, complicating protocol composition and adding extra latency to shard operations. (iii) High-concurrency cross-shard execution remains constrained by coordination-intensive commit paths, lock contention, and tight state-consensus coupling. Motivated by these observations, we adopt a sharding-centric design that scales throughput while keeping cross-shard coordination lightweight and ensuring predictable progress under adverse network conditions.

A. Related Work

Existing research has proposed a variety of solutions to address the security and performance limitations of sharded blockchains. Chainspace [6] adopts an object-based sharding model combined with a Hash-DAG representation, enabling transactions to explicitly encode their dependencies and achieve

atomic cross-shard commits through the S-BAC protocol. Monoxide [7] scales blockchains through asynchronous consensus zones and introduces eventual atomicity to complete cross-zone transactions without relying on a traditional two-phase commit protocol. Kronos [8] introduces a shard-managed buffer for the joint processing of valid and invalid transactions and leverages a batched attestation protocol to reduce the cost of inter-shard communication. SharDAG [9] proposes an adaptive sharding mechanism that uses runtime workload analysis to dynamically manage account placement, thereby improving scalability and reducing cross-shard overhead. Recent sharding protocols have also optimized cross-shard coordination. Liu et al. [10] improve cross-shard coordination through cross-shard Byzantine fault tolerance (CSBFT). It further runs multiple CSBFT instances in parallel to reduce confirmation latency. CHERUBIM [11] takes a different path. It uses a quadruple pipelined two-phase commit framework to overlap intra-shard and cross-shard phases. Despite these advances, large-scale deployments of sharded blockchains continue to expose three critical gaps:

1) *Deployment-Level Bottlenecks in Certification and View Changes*: In large-scale sharded deployments, certification and view-change handling can become significant bottlenecks. In particular, they can substantially increase the overhead required to generate verifiable shard outputs for cross-shard coordination and can undermine progress stability under frequent leader changes and network jitter. Despite progress in optimizing communication complexity and execution efficiency in BFT consensus protocols, structural limitations persist when scaling to large deployments in partially synchronous environments. These include inefficient signature verification, inconsistent state advancement across replicas, and misalignment between synchronization assumptions and protocol safety models [12], [13]. Traditional PBFT employs three-phase linear broadcasting and incurs a view-change communication complexity of $O(n^2)$. HotStuff [14], [15] improves this design by introducing a linear quorum certificate (QC) chain and a threshold signature scheme, reducing the normal-case complexity to $O(n)$ [16]. However, practical deployments may still experience nontrivial state-management overhead during frequent view changes, particularly when multiple speculative branches remain uncommitted. Fast-HotStuff [17] shortens the optimistic commit path through a dual-chain rule. However, view-change and timeout handling may still incur noticeable certificate-processing overhead under frequent timeouts. This overhead becomes more pronounced in weakly synchronous networks with unstable connectivity.

The lack of a lightweight and structured global state synchronization mechanism during view changes also leads to imbalanced state advancement across replicas [18], [19]. Although the chain-based structure guarantees eventual consistency, frequent view changes or network jitter often force replicas to maintain multiple uncommitted branches before consensus is reached [20]. Divergent timeout perceptions across replicas can induce sustained view-change oscillations, a pathological state in which the system cycles through leader views without committing new proposals. This behavior can severely compromise both liveness and throughput [21]. Sharded deployments benefit from mechanisms that simultaneously

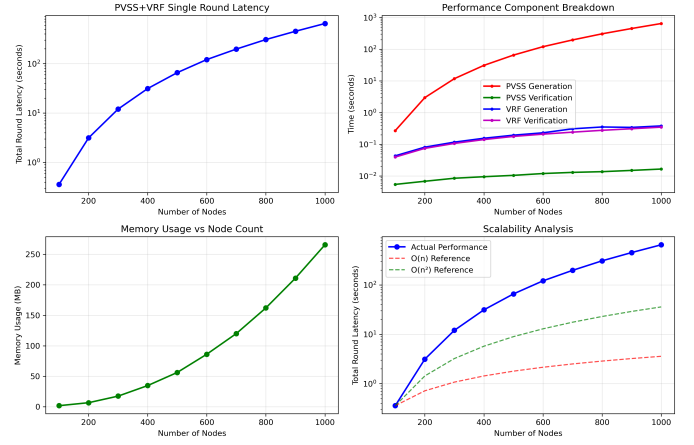


Fig. 1. Performance analysis of PVSS+VRF protocol.

reduce certificate-processing overhead and regulate progress synchronization during view changes, thereby helping to sustain stable throughput under adverse network conditions [22], [23].

These observations motivate an efficient mechanism for certification and view synchronization that reduces control-plane overhead while preserving stable progress under leader changes and network jitter.

2) *Deployment-Level Overhead of Decentralized Randomness and Reconfiguration*: Decentralized randomness beacons are often used in sharded blockchains to support committee selection and task scheduling. When such beacons are required, existing designs may incur substantial overhead and present nontrivial integration challenges in terms of scalability, security, and protocol composition [24], [25], [26], [27]. Traditional publicly verifiable secret sharing (PVSS) schemes such as RandHound can incur prohibitive communication complexity $O(n \cdot c^2)$, where c denotes the number of parties participating in a single PVSS instance. This cost arises from multi-round broadcasts, zero-knowledge proof verification, and secret reconstruction [28], [29]. Fig. 1 shows that latency departs from linear scaling and exhibits clear superlinear growth. At 1,000 nodes, both the per-epoch latency and the memory footprint exceed the 100-node case by more than an order of magnitude.

Moreover, many randomness protocols are designed as external modules, which introduce additional confirmation latency due to multi-block finality and coordination across protocol layers [30]. Hybrid designs attempt to embed PVSS into shard operations, yet they may still be constrained by coarse-grained synchronization delays. These delays hinder sub-second block production and introduce security-latency trade-offs.

This motivates a lightweight, deployment-friendly instantiation for publicly verifiable seeding that can be integrated into shard operations when desired, without making the core sharding workflow dependent on a standalone beacon.

3) *Structural Bottlenecks in Handling High-Concurrency Cross-Shard Transactions*: Existing sharded blockchains experience bottlenecks in high-concurrency cross-shard transaction processing. These bottlenecks arise primarily from serialized proposal limitations, atomic coordination overhead, and tight

TABLE I
COMPARISON OF SHARD-BASED CROSS-SHARD COORDINATION PROTOCOLS.

Protocol	Atomicity	Coordinator independence	Synchrony assumption	Intra-shard overhead	Cross-shard overhead	Verification cost
OmniLedger	Weak ¹	$\times^2$	Part. sync	$2k\mathcal{B}^4$	$O(b(\log b + \lambda))^5$	$2k \text{ sig}^6$
Chainspace	$\checkmark$	$\checkmark$	Part. sync	$2k\mathcal{B}$	$O(n^2b\lambda)$	$2k \text{ sig}$
RapidChain	$\checkmark$	$\triangle$	Part. sync	$k\mathcal{B}$	$O(n^2b\lambda)$	$k \text{ sig}$
Monoxide	Eventual	$\checkmark$	Prob. finality	$k\mathcal{B}$	$O(nb\lambda)$	1 Merkle proof
Kronos-VC	$\checkmark$	$\checkmark$	Part. sync / Sync	$k\mathcal{B}$	$O(nb\lambda)$	$k \text{ proofs} + 1 \text{ sig}$
Akaverse	$\checkmark^*$	$\checkmark$	Part. sync	$k\mathcal{B}$	$O(nb\lambda)$	$k \text{ BLS sig}$

Symbols. n : replicas per shard; b : cross-shard transactions per batch; λ : number of shards involved by a cross-shard transaction; k : distinct input shards of a transaction; $\mathcal{B}$: size of one certified artifact.

¹ Weak atomic progress: safety may hold, but completion depends on continued coordinator/client participation; Eventual atomicity: completion depends on eventually processed relay transactions.

² $\checkmark$ if cross-shard completion does not rely on a single designated coordinator to collect or forward certified cross-shard artifacts. $\times$ if such a role can delay or bias finalization by withholding messages.

³ This column reports the input-side certified-artifact footprint. A 2pc path needs two artifacts per input shard, giving $2k\mathcal{B}$; a one-shot validation or escrow path needs one, giving $k\mathcal{B}$. For Monoxide, the cost is counted per relay step.

⁴ This column counts cross-shard request/proof transfer for a batch of b transactions, excluding local BFT execution. The n^2 terms reflect all-to-all shard transfer, whereas the n terms are achieved by batching, coded transfer, or destination-grouped forwarding.

⁵ A 2PC path needs about $2k$ certified-artifact verifications, while a one-shot path needs about k . Batch-based designs add per-input inclusion/commitment proofs and one batch-level certificate. For Monoxide, the cost is per relay step; $k \leq \lambda$.

state-consensus coupling. The prevalent single-leader consensus model forces sequential transaction processing at monolithic proposers, creating throughput bottlenecks under heavy load.

In addition, the two-phase commit (2PC) mechanisms used to enforce cross-shard atomicity introduce contention through distributed state locking, which significantly degrades performance. Furthermore, the tight integration of execution and consensus phases requires redundant global state accesses due to the absence of intermediate caching, thereby increasing both latency and computational overhead. These constraints compound one another in high-frequency cross-shard invocation scenarios. As a result, no existing solution holistically optimizes the communication, execution, and data consistency planes simultaneously.

Such limitations call for a coordination-efficient cross-shard execution mechanism that reduces commit-path overhead, mitigates lock contention, and improves scalability under high concurrency.

B. Our Contributions

To address these limitations, we design Akaverse, a sharded blockchain system that preserves transactional consistency while reducing runtime overhead, making it suitable for high-throughput, large-scale execution environments. As summarized in Table I, Akaverse distinguishes itself from prior shard-based coordination protocols by simultaneously providing atomicity, strong fault tolerance, and reduced response complexity. The contributions of this paper are as follows:

1) Streamlined Threshold Consensus with EVS Gating:

Fast-Hotstuff protocol enable two-chain commits across view changes by relying on aggregate quorum certificate (AggQC), which incur $O(n)$ verification overhead and complicate the safety argument for recovery paths. Akaverse eliminates this dependency. It combines threshold BLS signatures with an enhanced view-synchronization gating rule (EVS) to confine two-chain fast commits to contiguous views, and reverts to

the standard three-chain rule upon timeout-triggered recovery. EVS binds newview to certified digests via threshold signatures, blocking stale forks without AggQC. This yields constant-size evidence with $O(1)$ verification cost, simplifying recovery safety.

2) Consensus-Embedded Threshold Randomness Beacon:

Akaverse introduces a threshold BLS-based randomness beacon embedded directly into the consensus pipeline. It provides publicly verifiable randomness for leader election, committee reconfiguration, and scheduling decisions without relying on external randomness infrastructure. Given a threshold key setup, each randomness round consists of signature-share dissemination followed by leader aggregation of any t valid shares. Under a leader-based aggregation model, this design requires one interaction round and $O(n)$ messages. It further incurs $O(t)$ group operations for aggregation and $O(1)$ constant-pairing verification for the final certificate. Each round derives a domain-separated seed from the committed chain state. The aggregated threshold signature serves as a randomness certificate, which is unpredictable before threshold completion and publicly auditable afterward. This strengthens fairness and resistance to role-selection manipulation while preserving the lightweight consensus path of Akaverse.

3) Single-Shard Multi-Leader Parallel Processing Scheme:

We propose a single-shard, multi-leader parallel execution scheme designed for high-throughput cross-shard transaction processing. Central to this scheme is a novel transaction coordination mechanism that integrates a global buffer pool. This mechanism organizes the cross-pool transaction lifecycle into three stages: (i) source-pool commitment; (ii) GBP-pool ordering; and (iii) destination-pool materialization. Crucially, the approach eliminates the need for traditional precommit and rollback interactions. Instead, it leverages local consensus at the source pool followed by GBP log ordering and destination-pool materialization, enabling low-latency, low-communication-overhead, and rollback-free execution of cross-shard transactions under high concurrency.

II. PROBLEM FORMULATION

In this section, we define the network and threat models that underlie the design of Akaverse.

A. System Model

We distinguish between the total number of replicas in the system and the size of each shard committee. Let N denote the total number of replicas in the network, and let n denote the number of replicas in each shard. We assume that the system is partitioned into S equal-sized shards.

1) *Network Model*: We assume a partially synchronous network model. Specifically, there exists a global stabilization time (GST) after which all messages exchanged among honest nodes are guaranteed to be delivered within a bounded delay of at most Δ . Before GST, message delays may be arbitrary. Strong synchrony is not required for correctness: safety is delay-independent, and liveness relies only on bounded message delay after GST. Node membership is dynamic: nodes may join or leave shards through a membership management protocol, as described in Section III.

2) *Node Model*: Each node in the system maintains its own local state and is assigned to a shard through a committee allocation mechanism. Shard membership remains fixed within each epoch. At the end of an epoch, the system performs partial intra-shard reconfiguration, allowing a subset of nodes to be dynamically replaced or reassigned.

B. Threat Model

We consider an adaptive Byzantine adversary $\mathcal{A}$ that can corrupt up to f replicas in each shard committee, where the shard size satisfies $n = 3f + 1$ as defined above. Corrupted replicas may arbitrarily deviate from the protocol, including generating invalid messages, delaying communication, and colluding to disrupt both intra-shard consensus and cross-shard execution. The system relies on randomized validator assignment to ensure that, with high probability, each shard committee maintains an honest majority. We assume authenticated channels and standard secure cryptographic primitives. Under this model, the system guarantees safety at all times and liveness under partial synchrony.

C. Sharded SMR Model and Cross-Pool Consistency

Akaverse follows a sharded state machine replication (SMR) model composed of multiple pool-level instances. Each transaction pool P_a maintains a locally totally ordered committed log, and all honest replicas in the same pool execute the same sequence of committed events. Cross-pool execution is modeled as a sequence of certified local events. A source-side event becomes eligible for cross-pool propagation only after it is finalized in the source-pool log. The corresponding target-side effect is applied only after the GBP-certified record is verified and committed through the destination pool's SMR log. Under this model, Akaverse guarantees pool-level total order and cross-pool causal consistency.

For each pool P_a , let $L_a = \langle e_a^1, e_a^2, \dots \rangle$ denote its committed SMR log. Each event $e \in L_a$ has a canonical local position $\chi(e) = (a, h, q, d)$, where a is the pool index, h is the committed block height, q is the event index within the block, and $d = H(e)$ is the event digest. The order of L_a is defined by the lexicographic order of $\chi(\cdot)$. Let P_g denote the GBP pool and L_g denote its committed log. A cross-pool transaction follows a certified forward path:

$$P_{src} : e_s \in L_{src} \longrightarrow P_g : e_g \in L_g \longrightarrow P_{dst} : e_t \in L_{dst}.$$

The source event e_s records the source-side effect, the GBP event e_g records the ordered cross-pool transfer record, and the target event e_t records the target-side effect.

For a GBP event e_g derived from a source event e_s , we define its source reference as:

$$\sigma(e_g) = (a_{src}, h_{src}, q_{src}, d_{src}),$$

which records the committed position of e_s in the source-pool log. The GBP admits such a record only after the source event has been locally finalized and certified.

For two GBP events e_g^i and e_g^j that originate from the same source pool and target the same destination pool, if:

$$\sigma(e_g^i) <_{\text{lex}} \sigma(e_g^j),$$

then e_g^i is ordered before e_g^j in the GBP log, and every honest replica in the destination pool applies the corresponding target-side event of e_g^i before that of e_g^j . For records from different source pools, Akaverse does not infer a causal order from their source logs. Within the same GBP block, their relative order is determined by the canonical ordering key $\kappa(\cdot)$. Across different GBP blocks, their relative order follows the committed order of the GBP log L_g . The resulting order is finally materialized through the destination pool's own SMR log L_{dst} .

The target-side effect of a cross-pool record is not applied outside consensus. After verifying the GBP commitment and the associated source-side evidence, the destination pool encodes the record as a local destination pool event and includes it in L_{dst} . The effect becomes ledger-visible only after this target-pool event is locally committed.

D. Cryptographic Primitives

Our protocol relies on the following standard cryptographic primitives.

1) *BLS Threshold Signatures*: We adopt a t -of- n BLS threshold scheme in which a DKG protocol generates the public key and the corresponding secret shares. Each party produces a signature share on a message, and any t shares can be aggregated into a unique global signature. The aggregated signature is verified using a single pairing check, and its security relies on bilinearity and the hardness of the (co-)CDH problem in bilinear groups.

2) *Hash Function*: We assume a collision-resistant hash function $H : 0, 1^* \rightarrow 0, 1^\lambda$. For any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the probability of finding a collision is negligible in the security parameter λ :

$$\Pr [(x, y) \leftarrow \mathcal{A}(1^\lambda) : x \neq y \wedge H(x) = H(y)] \leq \text{negl}(\lambda). \quad (1)$$

This standard collision-resistance property is the only requirement we impose on H .

3) *Merkle Tree*: A Merkle tree provides a cryptographic commitment that ensures verifiable data integrity. Given a dataset $D = d_1, \dots, d_n$, each element is hashed into a leaf node, and parent nodes are computed as $H(h_{\text{left}} \parallel h_{\text{right}})$ until the root rt is obtained. For each d_i , an authentication path hp enables efficient membership proofs. Verification requires only $\mathcal{O}(\log n)$ hash computations, while the structure guarantees tamper resistance under the collision resistance of H .

III. AKAVERSE PROTOCOL

Akaverse adopts a shard-based partitioning strategy in which nodes within the same shard run the Fast-HotStuff consensus protocol augmented with threshold signatures and an enhanced view synchronization (EVS) mechanism, thereby enforcing view-consistent progress during view changes. Each shard also maintains multiple transaction pools governed by independent leaders that execute parallel consensus pipelines. In addition, Akaverse introduces a GBP to batch cross-pool interactions into fixed windows, enabling deterministic finalization. Specifically, we propose a consensus-embedded randomness generation scheme driven by threshold signatures. This scheme enables Akaverse to generate unbiased, publicly verifiable per-view randomness directly within the protocol, without relying on external beacons. By leveraging distributed key management, multi-phase signature aggregation, and a synchronized QC mechanism, Akaverse tightly couples the randomness generation process with the consensus execution workflow, thereby enhancing both randomness quality and consensus security.

In the general workflow of Akaverse, a client submits a transaction request containing the account address, transaction amount, and signature information. Based on the account's designated transaction pool, the request is forwarded to the corresponding leader node, which verifies the signature validity, balance sufficiency, and cross-shard status. Upon successful verification, the transaction is packaged into a proposal block, and the leader node initiates a consensus broadcast in which BLS threshold signatures are used for collective vote aggregation. In addition, an enhanced view synchronization rule is introduced to ensure consistent progress across replicas and to eliminate divergence caused by delayed messages (see Section III-C).

After consensus is reached between the leader and its replica nodes, the block is officially committed to the blockchain. At this stage, the transaction is marked as submitted, the corresponding assets are deducted from the account, and a pending transfer marker is recorded on-chain as proof for subsequent processing. For requests involving cross-pool transactions, the confirmed transaction information is aggregated in batches, and the execution order is synchronized so that the transaction results can be dispatched to the corresponding destination pools for deterministic state updates (see Section III-D).

A. Node Initialization

System nodes participate in a DKG protocol to generate the public key PK and individual private key shares sk_i . Specifically, each node signs the genesis message m_0 to

produce a partial signature $\sigma_i = \text{Sign}(sk_i, m_0)$ and broadcasts it to the aggregation nodes. The aggregation nodes collect all valid signature shares and combine them into a threshold signature σ_0 . The randomness value for the first round is then computed as $r_0 = H(\sigma_0)$, which is embedded in the genesis block G_0 .

Subsequently, G_0 is disseminated across the network using a gossip protocol. Upon receiving G_0 , each node verifies the validity of σ_0 and computes the root shard identifier as $rootID_i = r_0 \bmod S$. Finally, G_0 is recorded in the local ledger, completing shard-state initialization.

B. Sharding Process

During the initialization phase, the system introduces a root shard that serves as a global coordinator for the sharding system. The root shard manages account address space partitioning and the selection of regular shard nodes but does not process regular transactions or maintain account states.

1) *Address-Space Partitioning for Accounts*: When a new account is created, the root shard generates a unique identifier for the account using its address hash value and computes the corresponding target shard ID. The target shard then processes transactions associated with the assigned account. This mechanism ensures that each account has a unique shard assignment and that subsequent transactions are consistently processed within the same shard.

2) *Shard Node Selection*: Each shard verifies the announced committee configuration before completing its internal structure update. A regular shard is responsible for processing transactions and maintaining shard state. Each shard operates several transaction pools, and every transaction is assigned to a designated leader. The leader coordinates with the replica nodes to execute the consensus protocol and propose new blocks.

In detail, pending transactions are partitioned into cross-shard and intra-shard transactions according to their account mappings. Each leader is responsible for proposing a block containing its assigned transactions. Transaction scheduling rules ensure that no conflicting transactions are processed simultaneously, thereby preventing state inconsistencies across parallel consensus instances. Leaders package the transactions in their assigned transaction pool into blocks and submit them to the replica group for consensus.

C. Intra-Shard Consensus Interface and Instantiation

In Akaverse, the sharding workflow and GBP-based cross-pool coordination do not depend on the internal phase structure of any specific consensus protocol. Instead, they rely on the externally verifiable semantics provided by intra-shard consensus. Specifically, they require a publicly verifiable proof of local finality for committed blocks, as well as a progress-consistent view-change rule that prevents honest replicas from diverging on the committed prefix.

Akaverse is inspired by the chained pipeline design of Fast-HotStuff and augments it with lightweight view synchronization for pool-level consensus. BLS signature aggregation is a mature technique and has been adopted in production blockchain systems, such as Ethereum's proof-of-stake (PoS) consensus

layer. Our design uses BLS-based compact certificates to support efficient pool-level consensus and certified cross-pool coordination in the sharded execution model. We instantiate compact quorum certificates using BLS threshold signatures. The design preserves $\mathcal{O}(n)$ communication complexity and a constant QC size, while its multi-transaction view-advancement strategy significantly accelerates view transitions and enables provably safe block finalization. This enhancement reduces fork occurrence and improves end-to-end confirmation latency within the chained consensus pipeline. The detailed execution process is described in the following subsections.

1) *Protocol Bootstrapping*: Leaders rotate according to a monotonically increasing view number $v \in \mathbb{N}$, and QCs are represented in a unified form as follows:

$$QC = \langle type, v, y, \Sigma \rangle, \quad (2)$$

where $type \in \{prepare, commit, \dots\}$ denotes the phase tag, v is the current view number. Each QC certifies a block digest y . We write $QC.block$ to denote this digest, and use y as shorthand in the signature context. We assume a (t, n) BLS threshold setting with a group public key PK and per-replica share verification keys $\{VK_i\}_{i=1}^n$. Each replica i holds a threshold signing share sk_i and produces signature shares that are verified using VK_i before aggregation. Threshold signatures are verified using the group key PK . The symbol Σ denotes a BLS threshold signature bound to both the current view and the certified payload. Each replica i produces a phase vote by domain-separated signing of the triple $(type, v, y)$:

$$\sigma_i = \text{SignShare}_{sk_i}(H(type \parallel v \parallel y)).$$

The verification process requires only a single check on the view-bound message:

$$\text{BLS.Verify}(PK, \Sigma, H(type \parallel v \parallel y)). \quad (3)$$

Each replica persists the highest-view certificate it has ever accepted, denoted as $HighQC_i$. It serves as the justification for the next proposal and is used to update the local lock state for safety.

2) *Enhanced View Synchronization*: EVS acts as a front-door gate for all handlers (Fig. 2). A replica processes only messages for its current view, discards messages from older views, and buffers messages for the immediate next view. Messages beyond the immediate next view are not processed by the EVS gate.

The procedure follows Algorithm 1. On a local timeout at view v , replica i prepares a *viewchange* message for view $v+1$ under a fixed signing domain. It sets the bound digest h_i to the block digest certified by its current $HighQC_i$, i.e., $h_i = HighQC_i.block$. The replica then signs $H(\text{viewchange} \parallel v+1 \parallel h_i)$ using its threshold signing share and sends $\langle \text{viewchange}, v+1, i, HighQC_i, h_i, \sigma_i^{vc} \rangle$ to the next leader, where $\sigma_i^{vc} = \text{SignShare}_{sk_i}(H(\text{viewchange} \parallel v+1 \parallel h_i))$.

When the new leader receives *viewchange* messages, it groups valid messages by their bound digest. For a group with at least $2f+1$ valid messages from distinct replicas carrying the same digest h^* , every carried certificate $HighQC_j$ must satisfy $h^* = HighQC_j.block$. The leader then aggregates the

Algorithm 1 View-Synchronized Fast Consensus

Input: Threshold $2f+1$, group key PK
Output: Committed blocks and updated local consensus state

```

1: State:  $CurView_i, HighQC_i, LockQC_i, Buf_i, Recovered_i$ 
2: function EVS( $m$ )
3:   if  $m.v < CurView_i$  or  $m.v > CurView_i + 1$  then
4:     return drop
5:   else if  $m.v = CurView_i + 1$  then
6:     if  $m.type = newview$  and  $VALIDNEWVIEW(m)$  then
7:       install  $m.HighQC^*$ ;  $CurView_i \leftarrow m.v$ 
8:        $Recovered_i \leftarrow \text{true}$ 
9:     else if  $VALIDNEXTPROPOSAL(m)$  then
10:       $CurView_i \leftarrow m.v$ ;  $DISPATCH(m)$ 
11:   else
12:      $Buf_i \leftarrow Buf_i \cup \{m\}$ 
13:   else
14:      $DISPATCH(m)$ 
15: procedure TIMEOUT
16:    $v' \leftarrow CurView_i + 1$ ;  $h_i \leftarrow \text{block}(HighQC_i)$ 
17:    $\sigma_i^{vc} \leftarrow \text{SHARESIGN}(\text{viewchange}, v', h_i)$ 
18:   send  $\langle \text{viewchange}, v', i, HighQC_i, h_i, \sigma_i^{vc} \rangle$  to  $LEADER(v')$ 
19: procedure MAKENEWVIEW( $\mathcal{M}, v'$ )
20:   group valid viewchange messages by  $h_j = \text{block}(HighQC_j)$ 
21:   if some group  $h^*$  has  $2f+1$  valid shares from distinct replicas then
22:     choose highest-view  $HighQC^*$  with  $\text{block}(HighQC^*) = h^*$ 
23:      $\Sigma^{vc} \leftarrow \text{AGGSHARES}(\text{viewchange}, v', h^*)$ 
24:     broadcast  $\langle newview, v', HighQC^*, \Sigma^{vc} \rangle$ 
25: procedure PREPARE( $v', HighQC^*$ )
26:   let  $B_p$  be the block certified by  $HighQC^*$ 
27:   propose  $B_{v'} = \langle \text{Block}, H(B_p), v', \text{height}(B_p)+1, \text{txs} \rangle$ 
28:   if  $B_{v'}.parent = \text{block}(HighQC^*)$  and  $LOCKOK(B_{v'})$  then
29:     replicas send prepareVote shares on  $H(B_{v'})$ 
30:   if  $2f+1$  valid shares are collected then
31:     form  $QC_{prepare}(B_{v'}) = \langle \text{prepare}, v', H(B_{v'}), \Sigma^{prep} \rangle$ 
32: procedure COMMITCHECK( $B_{v'}$ )
33:   let  $B_p$  be the direct parent of  $B_{v'}$ 
34:   let  $QC_p \leftarrow QC_{prepare}(B_p)$  and  $QC_{v'} \leftarrow QC_{prepare}(B_{v'})$ 
35:   if  $Recovered_i = \text{false}$  and  $QC_p$  exists and  $QC_{v'}$  exists then
36:     if  $B_{v'}$  directly extends  $B_p$  and  $QC_p, QC_{v'}$  are view-consecutive then
37:       commit  $B_p$  // restricted two-chain fast path
38:   return
39:   apply the standard three-chain commit rule

```

corresponding signature shares over $H(\text{viewchange} \parallel v+1 \parallel h^*)$ to obtain the *viewchange* threshold signature Σ^{vc} . It then broadcasts *newview*:

$$\langle newview, v+1, HighQC^*, \Sigma^{vc} \rangle \quad (4)$$

Replicas recompute h^* and verify Σ^{vc} against the *viewchange* domain.

$$\text{BLS.Verify}(PK, \Sigma^{vc}, H(\text{viewchange} \parallel v+1 \parallel h^*)) \quad (5)$$

If the check succeeds, replica i advances to view $v+1$ and updates $HighQC_i$ to $HighQC^*$ if $HighQC^*$ carries a higher view number. Since the view is entered through a verified *newview* message, replica i sets $Recovered_i \leftarrow \text{true}$. The flag is set only for timeout-triggered view recovery and remains unset during normal leader rotation. It is cleared only after replica i locally completes two consecutive normal consensus cycles, each consisting of receiving a valid proposal from the deterministic leader, casting a *prepare* vote, and accepting the formed $QC_{prepare}$, without triggering timeout-based recovery.

If replica i has cast a *prepare* vote for block B_v in view v , it may enter view $v+1$ upon receiving a valid proposal from $Leader(v+1)$ that carries $QC_{prepare}(B_v)$ as the parent justification and satisfies the locking rule. This transition does not involve a *newview* message and does not set $Recovered_i$.

In each view, replicas maintain a local safety lock and accept only proposals justified by a valid QC and allowed by

the locking rule. The EVS synchronization rules are defined as follows:

- **Old-view discard.** Drop any message with $msg.v < CurView_i$.
- **Next-view admission.** For $msg.v = CurView_i + 1$, a valid *newview* may be verified immediately and used to enter the next view with $Recovered_i \leftarrow \text{true}$. A valid proposal from $Leader(CurView_i + 1)$ carrying a valid $QC_{prepare}$ for its parent block may also advance the replica through normal leader rotation without setting $Recovered_i$. Other ordinary next-view messages are buffered.
- **Single-step processing.** The replica processes current-view messages with $msg.v = CurView_i$ and the admissible next-view messages above. Messages with $msg.v > CurView_i + 1$ are rejected by the EVS gate.

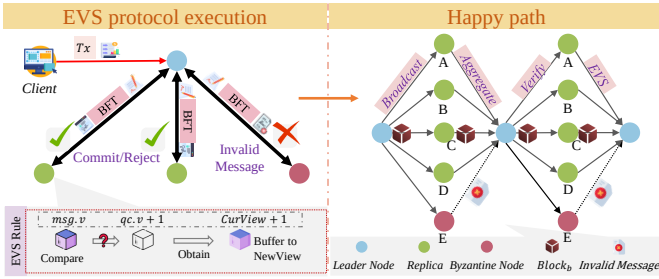


Fig. 2. Processing of the EVS protocol

3) *Happy Path Phase*: Assume that view v' is active after EVS admission. The fast path in this subsection is enabled only when $Recovered_i = \text{false}$, meaning that the current view was entered through normal leader rotation rather than timeout-triggered recovery. If $Recovered_i = \text{true}$, replicas may still process valid proposals, but commit decisions follow the three-chain fallback rule.

In the failure-free contiguous path, the leader of view v' proposes a block extending the parent justified by the latest valid prepare QC.

We denote the justification certificate used by the leader in view v' as $HighQC^*$, which justifies the parent block selection for the next proposal. $HighQC^*$ is typically a *prepare* certificate for a parent block B_p , written uniformly as $HighQC^* = QC_{prepare}(B_p)$. On the consecutive happy path, B_p is the block proposed in view $v' - 1$. With v' active, the leader proposes a new block that extends the parent justified by $HighQC^*$. Let $h_{parent} = H(B_p)$ denote the parent block digest. The proposal carries $B_{v'}$ together with $HighQC^*$ as justification, as follows:

$$B_{v'} = \langle Block, h_{parent}, v', txs, \dots \rangle, \quad (6)$$

The leader attaches its individual proposal signature:

$$\text{sig}_\ell^{prop} = \text{Sign}_{sk_\ell^{prop}}(H(\text{prepProp} \parallel v' \parallel H(B_{v'}))). \quad (7)$$

Upon receiving the prepare message, each replica first applies the EVS synchronization rules and then verifies: (i) the sender is $Leader(v')$ and the leader's proposal signature sig_ℓ^{prop} is valid; (ii) the carried $HighQC^*$ is valid and justifies the declared

parent; (iii) parent-child consistency, namely that the parent digest carried by $B_{v'}$ equals $HighQC^*.block = H(B_p)$; and (iv) the proposal satisfies the local locking rule.

If all checks pass, replica i casts a **prepare** vote using its threshold share:

$$\sigma_i^{prep} = \text{SignShare}_{sk_i}(H(\text{prepare} \parallel v' \parallel H(B_{v'}))). \quad (8)$$

When the leader gathers at least $2f + 1$ valid votes from distinct replicas on the same message, it aggregates them into:

$$QC_{prepare}(B_{v'}) = \langle \text{prepare}, v', H(B_{v'}), \Sigma^{prep} \rangle. \quad (9)$$

By the chained HotStuff safety rule, a block is irrevocably committed once a three-chain of consecutively justified blocks is formed. Akaverse additionally uses a restricted two-chain fast commit in the failure-free contiguous path. A replica may commit $B_{v'}$ by the two-chain rule only if all of the following conditions hold: (i) $QC_{prepare}(B_{v'})$ and $QC_{prepare}(B_{v'+1})$ are both valid; (ii) $B_{v'+1}$ directly extends $B_{v'}$; (iii) the two QCs are formed in consecutive views v' and $v'+1$; and (iv) $Recovered_i = \text{false}$, meaning that no timeout-triggered recovery intervenes in this fast path. If any condition fails, the replica applies the standard three-chain commit rule.

Operationally, once $QC_{prepare}(B_{v'+1})$ is formed, the leader may disseminate the triggering certificate as a commit notice. Each replica verifies the threshold signature on $H(\text{prepare} \parallel v'+1 \parallel H(B_{v'+1}))$ and checks that this certificate forms a direct parent-child two-chain with $QC_{prepare}(B_{v'})$ in consecutive views.

If the checks pass and $Recovered_i = \text{false}$, the replica marks $B_{v'}$ as committed and updates its committed pointer. Otherwise, the protocol falls back to the standard three-chain commit rule, preserving safety under recovered or non-contiguous views.

4) *Unhappy Path Phase*: If progress is not observed before the local timeout in view v , replicas trigger a view change as follows:

- **Send viewchange.** Replica i computes h_i and sends $\langle \text{viewchange}, v+1, i, HighQC_i, h_i, \sigma_i^{vc} \rangle$ to $Leader(v+1)$, where σ_i^{vc} is a threshold-share signature over $H(\text{viewchange} \parallel v+1 \parallel h_i)$. Once an honest replica sends a viewchange message for view $v+1$, it enters a view-changing state and will not cast any new prepare vote for views $v' \leq v$.
- **Collect and broadcast newview.** The new leader verifies the carried certificates, checks h_i , and groups valid *viewchange* messages by their bound digest. For a group with at least $2f+1$ valid shares from distinct replicas carrying the same digest h^* , the leader selects a highest-view valid $HighQC^*$. It aggregates the corresponding shares over $H(\text{viewchange} \parallel v+1 \parallel h^*)$ to obtain Σ^{vc} and broadcasts $\langle \text{newview}, v+1, HighQC^*, \Sigma^{vc} \rangle$.
- **View update and continuation.** Upon successful verification, replica i advances to view $v+1$, updates $HighQC_i$ to $HighQC^*$, and sets $Recovered_i \leftarrow \text{true}$. By EVS, messages from views lower than $v+1$, including late proposals or votes for view v , are dropped, while buffered messages for view $v+1$ are released. The new leader

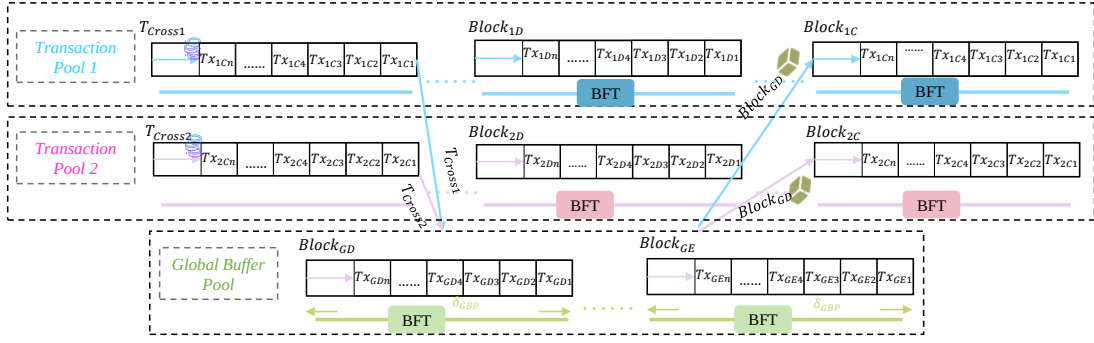


Fig. 3. Processing of GBP Protocol

must propose a block extending the parent justified by $HighQC^*$ and satisfying the locking rule.

D. Transaction Pool Organization

While the enhanced consensus mechanism ensures safety and synchronization within each shard, it does not fully address latency in cross-shard transaction finalization. Akaverse therefore adopts a multi-pool parallel pipeline architecture that enables concurrent transaction execution and deterministic finalization across interdependent pools.

1) Single Pool Transaction Sequence: For a single pool, committed transactions are ordered by consensus into a unique sequence. Committed blocks have monotonically increasing heights, and all honest replicas execute an identical transaction sequence.

The detailed design is presented in the following section.

2) Global Coordination Phase: Cross-pool transactions are handled by the GBP, which enforces certified ordering and step-level finalization across transaction pools. After a regular pool commits a block, it extracts the cross-pool output records generated by the committed transactions and submits these records to the GBP together with the corresponding source-pool commitment evidence.

The GBP admits only certified cross-pool records, verifies the source-pool certificate, checks replay status, and orders the admitted records through its own pool-level SMR log.

For cross-pool transactions, Akaverse adopts step-level atomicity with rollback-free forward execution. A cross-pool transaction is executed as a sequence of locally committed events across the source pool, the GBP pool, and the destination pool. The source-side effect becomes ledger-visible only after source-pool finality. The cross-pool record becomes deliverable only after it is committed in the GBP log. The target-side effect becomes ledger-visible only after the corresponding GBP-committed record and source-side evidence are verified and the target-side event is committed by the destination pool. If a later event fails before local commitment, it produces no ledger-visible effect. However, previously committed events are not rolled back and remain valid inputs for subsequent processing.

3) Global Buffer Pool: In each shard, Akaverse designates one special transaction pool as the GBP. The GBP is a dedicated

pool-level SMR instance for cross-pool coordination. It runs the same consensus pipeline as regular pools, but its state machine processes certified cross-pool records rather than ordinary user transactions. The GBP maintains a committed SMR log for certified cross-pool records. Its state consists of admitted records indexed by their unique identifiers and source references. The GBP operates in fixed scheduling windows of length δ . During each window, it collects certified cross-pool output records derived from committed source-pool blocks. At the end of the window, the GBP leader proposes a GBP block B_g that contains the admitted records ordered according to the deterministic GBP ordering rule. The block B_g is then committed by the GBP pool through its consensus pipeline.

A GBP block is valid only if every included record satisfies three conditions. First, source-proof admissibility holds. Each record must carry valid source-pool commitment evidence, so the GBP never admits a source-side effect that has not been locally committed. Second, replay safety holds. Each record is uniquely identified by its transaction identifier and source reference, and a record that has already been admitted or delivered cannot be processed again. Third, deterministic ordering holds. Within each GBP block, all admitted records are ordered by the canonical key:

$$\kappa(r) = (a_{src}, h_{src}, q_{src}, d_{src}, a_{dst}),$$

where a_{src} is the source-pool index, h_{src} is the committed source-pool height, q_{src} is the record index within the source block, d_{src} is the source-side record digest, and a_{dst} is the destination-pool index. For records from the same source pool to the same destination pool, this ordering preserves their source-side committed order. For records from different source pools, the canonical key provides a deterministic serialization within the GBP block. Across different GBP blocks, the relative order follows the committed order of the GBP log.

Once a GBP block is committed, its records become eligible for destination-pool materialization. A destination pool processes a cross-pool record only after verifying the GBP-pool commitment, the inclusion of the record in the committed GBP block, and the associated source-side evidence.

The GBP serves as a shared ordering point for certified cross-pool records, but it does not directly materialize target-side effects. Each destination pool commits the resulting target-side events after completing the required verification. We discuss the alternative direct-verification model in Section IV-B.

4) *Processing of Multi-Transaction Pool Operation*: Fig. 3 illustrates the GBP-pool coordination path, where certified cross-pool output records are extracted after source-pool commitment, ordered in the GBP log, and materialized through destination-pool SMR logs.

Step 1: Source-pool commitment and output extraction. In each source pool P_i , transactions are first ordered and committed through the pool-level consensus. After a regular pool P_i commits a block B_i^h , it extracts the cross-pool output records from the block. Let $\text{Out}(B_i^h)$ denote the set of output records generated by B_i^h . The cross-pool output set is defined as:

$$\mathcal{R}_i^h = \{r \in \text{Out}(B_i^h) \mid r.\text{dst} \neq P_i\}.$$

Each record $r \in \mathcal{R}_i^h$ is submitted to the GBP together with the source-pool commitment evidence and inclusion proof.

In Step 1, the source-pool commitment is recorded in a transfer-safe manner. Once a transaction is confirmed in the source pool, the debit effect is represented in the source ledger as a deterministic pending-out marker, which can be viewed as an escrowed transfer state. The source pool updates its state by reducing the spendable balance and creating a pending-out record. This record is committed under local consensus and therefore serves as a verifiable proof of source-side finality. This mechanism requires no cross-pool locking interaction and does not stall the source-pool pipeline.

Nevertheless, it still enables atomicity because the pending-out record is resolved only after the corresponding cross-pool record has been committed in the GBP log and materialized in the destination-pool log. Thus, the resolution follows the certified forward path from source-pool finality to GBP-log ordering and finally to destination pool commitment.

Step 2: GBP pool ordering. The GBP pool collects certified cross-pool records within a scheduling window δ . For each admitted record, the GBP verifies the source-pool QC and the source reference $\sigma(\cdot)$. The GBP leader then proposes a GBP block B_g containing the admitted records, ordered according to the deterministic GBP ordering rule. After B_g is committed by the GBP-pool consensus, the ordered records become eligible for delivery to their destination pools.

Step 3: Destination-pool materialization. After a GBP block B_g is committed in the GBP pool, each destination pool P_j derives its projection:

$$\mathcal{R}_j(B_g) = \{r \in B_g \mid r.\text{dst} = P_j\}.$$

For each record $r \in \mathcal{R}_j(B_g)$, replicas in P_j verify the GBP-pool commitment, the inclusion of r in the committed GBP block, the associated source-pool evidence, and the replay status of r . The destination pool leader then packages the verified records into a local block B_j and runs the consensus to commit B_j . Only after B_j is locally committed are the corresponding target-side balance or state updates applied.

5) *Cross Pool Transaction Sequence*: A cross-pool transfer becomes executable only after its source-side cause has been fully committed and propagated. The original transaction is first committed in the source pool under consensus, and is then aggregated by GBP into a derived cross-pool transfer record. This record must be admitted, ordered, and committed in the

GBP pool before it can be delivered to the destination pool. The destination pool applies the corresponding effect only after verifying the committed cross-pool record together with the associated committed chain from the source pool.

E. Correctness

Correct cross-pool execution is ensured through height-ordered processing, gap-aware synchronization, and replay protection. Cross-pool transfers are ordered by verified source-side heights rather than network arrival order. A transfer becomes eligible for proposal only after its corresponding source-side height has been locally verified, so out of order delivery does not affect the execution sequence. If prerequisite source-side blocks are missing, the block manager detects the gap, synchronizes the missing blocks, and buffers the transfer until all prerequisite blocks are available and verified. To prevent duplicate delivery, each transfer carries a unique digest that is checked against persistent storage before processing, while propagation is constrained by monotonic source-side height progression. These mechanisms preserve execution order, prevent premature processing, and ensure at-most-once application of each cross-pool effect.

IV. SECURITY AND COMPLEXITY ANALYSIS

A. Security Analysis

The Akaverse protocol provides robust security under the Byzantine fault model through a layered design. Fork prevention is achieved by using BLS threshold signatures to cryptographically bind replicas to a single block per view, reinforced by protocol rules that prevent honest nodes from signing conflicting blocks. View synchronization further ensures that the chain extends only from the highest certified block. For cross-pool transactions, consistency is maintained through pool-level finality, GBP-log ordering, and destination-pool materialization. Akaverse does not impose a single global order over all transactions. Instead, each pool maintains its own committed SMR log, while the GBP log provides a deterministic order for admitted cross-pool records that is consistently observed by destination pools.

Theorem IV.1 (Fork Prevention). *Under the threshold-vote aggregation model, Akaverse prevents same-view forks through quorum intersection and domain-separated vote binding. For any view v , no two conflicting blocks can both obtain valid QC_{prepare} certificates in view v . Equivalently, if $QC_{\text{prepare}}(B_1)$ and $QC_{\text{prepare}}(B_2)$ are both valid in view v , then $H(B_1) = H(B_2)$.*

Proof. We prove the theorem by contradiction. Suppose that, in some view v , two conflicting blocks B_1 and B_2 both obtain valid prepare QCs, where $H(B_1) \neq H(B_2)$.

Step 1: QC structure and message binding. In our protocol, a prepare QC for a block B is represented as $(\text{prepare}, v, H(B), \Sigma)$. It is generated by aggregating at least $2f+1$ valid threshold-signature shares from distinct replicas over the same domain-separated message m . Under

the contradiction assumption, there exist two valid prepare QCs in the same view v :

- $QC_{\text{prepare}}(B_1) = \langle \text{prepare}, v, H(B_1), \Sigma_1 \rangle$;
- $QC_{\text{prepare}}(B_2) = \langle \text{prepare}, v, H(B_2), \Sigma_2 \rangle$.

Since $H(B_1) \neq H(B_2)$, the bound messages $m_1 = H(\text{prepare} \parallel v \parallel H(B_1))$ and $m_2 = H(\text{prepare} \parallel v \parallel H(B_2))$ are different. Let S_1 and S_2 denote the existential signer sets of QC_1 and QC_2 , respectively, i.e., the sets of distinct replicas whose valid signature shares are sufficient to generate the corresponding threshold certificates. These sets are not fields of the final QCs and need not be recoverable from the threshold signatures. Their existence follows from threshold-signature unforgeability and the requirement that a valid threshold QC requires at least $2f + 1$ valid shares from distinct replicas. Therefore, by the validity of the two prepare QCs, we have $|S_1| \geq 2f + 1$ and $|S_2| \geq 2f + 1$. The domain-separated threshold signature binds each threshold certificate Σ_i to its certified message m_i , so a valid certificate for m_i cannot be reused to certify any different message $m_j \neq m_i$. In particular, a valid prepare QC for B_i certifies exactly the tuple $(\text{prepare}, v, H(B_i))$.

Step 2: Honest-set intersection and contradiction. Let $\mathcal{H}$ denote the set of honest replicas and $\mathcal{F}$ denote the set of byzantine replicas. In our model, $n = 3f + 1$, $|\mathcal{F}| \leq f$. Since S_1 and S_2 are two signer sets of valid threshold prepare QCs, by the threshold requirement:

$$|S_1| \geq 2f + 1, \quad |S_2| \geq 2f + 1. \quad (10)$$

Therefore, according to the quorum-intersection property:

$$|S_1 \cap S_2| \geq |S_1| + |S_2| - n \geq (2f+1) + (2f+1) - (3f+1) = f+1. \quad (11)$$

Since at most f replicas are byzantine, the intersection $S_1 \cap S_2$ must contain at least one honest replica. Thus, there exists an honest replica

$$r^* \in S_1 \cap S_2 \cap \mathcal{H}. \quad (12)$$

Since $r^* \in S_1 \cap S_2$, replica r^* must have contributed a valid prepare-share signature to QC_1 over m_1 and another valid prepare-share signature to QC_2 over m_2 . Thus, r^* must have signed both $H(B_1)$ and $H(B_2)$ in the same view v .

However, an honest replica issues at most one *prepare* vote in a given view. After voting for a valid prepare proposal in view v , it refuses to vote for any conflicting proposal in the same view. This contradicts the existence of the honest replica r^* that contributed to both threshold certificates. Therefore, the initial assumption is impossible. Hence, in any view v , no two conflicting blocks can both obtain valid QC_{prepare} certificates. $\square$

Lemma A1 (View synchronization with digest consistency). *If an honest replica accepts a newview certificate for view $v+1$, then the certificate proves that at least $2f+1$ distinct replicas issued valid viewchange shares over the same certified block digest h^* . Moreover, the carried $HighQC^*$ satisfies $\text{block}(HighQC^*) = h^*$.*

Proof. A replica accepts a *newview* message only after verifying the threshold signature Σ^{vc} over $H(\text{viewchange} \parallel v+1 \parallel$

$h^*)$. By threshold-signature unforgeability, such a certificate can be generated only from at least $2f+1$ valid signature shares from distinct replicas over the same message. In EVS, each viewchange share is produced over $H(\text{viewchange} \parallel v+1 \parallel h_i)$, where $h_i = \text{block}(HighQC_i)$. Therefore, all shares aggregated into Σ^{vc} bind the same certified block digest h^* .

The accepting replica also verifies the carried $HighQC^*$ and recomputes $h^* = \text{block}(HighQC^*)$. Hence, the accepted *newview* certificate is bound to the block digest certified by $HighQC^*$. $\square$

Lemma A2 (Stale Fork Non-Extension under EVS). *Once a valid newview certificate for view $v+1$ is formed, any stale fork from view v or any lower view cannot obtain a new valid QC_{prepare} . Moreover, after an honest replica enters the recovered view through this newview certificate, it will not vote for proposals that are inconsistent with the certified prefix installed by $HighQC^*$.*

Proof. Consider a valid *newview* message for view $v+1$: $\langle \text{newview}, v+1, HighQC^*, \Sigma^{vc} \rangle$. By the EVS verification rule, Σ^{vc} is a valid threshold certificate over: $H(\text{viewchange} \parallel v+1 \parallel h^*)$, where the accepting replica recomputes: $h^* = \text{block}(HighQC^*)$. Thus, the *newview* certificate is bound to the block digest certified by $HighQC^*$.

1. View-change quorum removes honest voters from stale views. By the protocol rule, once an honest replica sends a viewchange message for view $v+1$, it enters a view-changing state and will not cast any new prepare vote for views $v' \leq v$. Therefore, after the valid *newview* certificate is formed, at least $f+1$ honest replicas have already withdrawn from the voting pool of view v and lower views.

2. Stale forks cannot gather a prepare quorum. A valid QC_{prepare} requires at least $2f+1$ valid prepare votes from distinct replicas. After the $f+1$ honest view-change senders stop voting for views $v' \leq v$, at most $n - (f+1) = (3f+1) - (f+1) = 2f$ replicas can still contribute votes to any stale proposal from view v or lower. This number is strictly smaller than the quorum threshold $2f+1$. Hence, no stale fork from view v or any lower view can obtain a new valid QC_{prepare} once the valid *newview* certificate for view $v+1$ has been formed.

3. Recovered-view proposals must extend the installed prefix. When an honest replica accepts the valid *newview* message, it installs $HighQC^*$ as its highest evidence, advances to view $v+1$, and sets $Recovered_i \leftarrow \text{true}$. Under EVS, it drops messages from views lower than its current view. In the recovered view, it votes only for proposals that carry a valid justification QC, extend the parent certified by $HighQC^*$, and satisfy the local locking rule. Therefore, a proposal on a fork inconsistent with the certified prefix of $HighQC^*$ fails the parent-consistency or locking-rule check at honest replicas.

Consequently, stale forks may still be delivered by Byzantine replicas or delayed by the network, but they cannot obtain new honest prepare votes sufficient to form a valid QC_{prepare} , nor can they be extended by honest replicas after recovery. $\square$

Theorem IV.2 (Protocol Consistency). *Given a cross-pool transaction T_x proposed within the single-shard, multi-transaction pool protocol, all honest nodes in both the source pool and the destination pool involved in T_x , under the coordination of the GBP mechanism, achieve eventual consistency through the following properties:*

- 1) **Pool-level log consistency.** *All honest replicas in the same pool agree on the same committed SMR log.*
- 2) **GBP-log order consistency.** *All admitted cross-pool records are ordered through the committed GBP log, and this order is consistently observed by destination pools.*
- 3) **Destination-pool state convergence.** *Each destination pool executes verified GBP records through its own SMR log and deterministically reaches the same final state.*

We guarantee the above cross-pool state consistency by proving the three lemmas below.

Lemma A3 (Local Pool Finality). *For any transaction pool P_i , if a block B_i carrying a valid QC is committed, then the execution results of all transactions in B_i become irreversible within P_i .*

Proof. According to Theorem IV.1, blocks at the same height cannot coexist. Once a block is committed, a node rejects any proposal that conflicts with it. $\square$

Lemma A4 (GBP-Log Order Consistency). *Suppose two cross-pool records r_i and r_j are committed in the GBP log. All honest replicas observe the same committed order of r_i and r_j . Moreover, if r_i and r_j originate from the same source pool and target the same destination pool, and $\sigma(r_i) <_{\text{lex}} \sigma(r_j)$, then r_i precedes r_j in the GBP log and is applied earlier at the destination pool.*

Proof. The GBP is a pool-level SMR instance. Therefore, all honest replicas agree on the same committed GBP log. A GBP block is valid only if each included record carries valid source-pool evidence and the records are ordered according to the canonical key $\kappa(\cdot)$. This rule preserves the source-side order for records from the same source pool and gives a deterministic serialization for records from different source pools within the same GBP block. Across different GBP blocks, the relative order follows the committed order of the GBP log.

For records from the same source pool to the same destination pool, the ordering rule follows the source reference $\sigma(\cdot)$. Thus, the source-side order is preserved in the GBP log. The destination pool accepts only committed GBP records and commits the corresponding target-side events through its own SMR log. Therefore, all honest replicas in the destination pool apply these records in the same order. $\square$

Lemma A5 (Destination-Pool State Convergence). *For any destination pool P_j , let $\mathcal{R}_j$ be the ordered projection of committed GBP records addressed to P_j . If all honest replicas in P_j start from the same committed state and commit the corresponding target-side events through the same local SMR log, then they deterministically converge to the same final state.*

Proof. By Lemma A3, all honest replicas observe the same committed GBP-log order and derive the same projection $\mathcal{R}_j$ for P_j . Each record in $\mathcal{R}_j$ is processed only after its GBP commitment, source-pool evidence, inclusion proof, and replay status are verified. The destination pool then packages the verified records into local target-side events and commits them through its own SMR log. Since all honest replicas in the same pool agree on the same committed local log and execute the same deterministic state-transition function from the same initial state, they obtain the same final state. A faulty proposal with a different state root cannot obtain a valid QC and is rejected. $\square$

Theorem IV.3 (Protocol Atomicity). *The atomicity semantics of Akaverse consist of intra-pool atomicity and cross-pool atomicity. For any execution within a single pool, Akaverse guarantees all state updates take effect iff the execution is locally committed. For any cross-pool execution, Akaverse guarantees each step becomes effective only upon local commitment, and any committed step remains valid despite later failures.*

We formalize and prove the two constituents in Lemma A6 and Lemma A7.

Lemma A6 (Intra-pool atomicity). *For any execution confined to a single pool in Akaverse, all state updates become effective if and only if the execution is locally committed. If the execution aborts before local commitment, none of its state updates takes effect.*

Proof. Before local commitment, an execution produces no externally visible state update; if it aborts, the system discards all tentative updates, leaving the committed state intact. Only upon successful commitment are all of its state updates installed in the committed state as a single indivisible unit. Therefore, single-pool execution provides all-or-nothing atomicity. $\square$

Lemma A7 (Cross-pool atomicity). *For any cross-pool execution in Akaverse, each step s_i takes effect exclusively upon local commitment within its designated pool. Once committed, the effects of s_i are irrevocable and constitute legitimate inputs to all downstream cross-pool steps. Consequently, subsequent step failures cannot retroactively invalidate s_i .*

Proof. A cross-pool execution in Akaverse comprises an ordered sequence of steps committed locally across pools. By Lemma A6, each step satisfies intra-pool all-or-nothing atomicity: no state update is externally visible prior to local commitment, and upon commitment all updates are applied indivisibly. Consequently, a step becomes effective exclusively upon local commitment in its corresponding pool, and subsequent cross-pool processing can observe only effects produced by previously committed steps. Consider an arbitrary step s_i in this sequence. We distinguish two cases based on its commitment boundary.

Case 1 (Failure before commitment). If s_i aborts before achieving local commitment, it leaves no persistent effect in its pool. Consequently, no downstream step can observe any output from s_i , and the cross-pool execution proceeds as if s_i had never been submitted.

Case 2 (Failure after commitment). Once s_i is locally committed, its effects are installed in the committed state of its pool. Even if subsequent steps fail, such later failures cannot retroactively invalidate s_i . Thus, the committed effects of s_i remain valid and may serve as inputs to downstream cross-pool processing.

Since failures can only affect uncommitted steps, already-committed steps are irrevocable across pools. Therefore, cross-pool execution in Akaverse satisfies step-level atomicity. $\square$

B. Complexity Analysis

1) *Consensus Protocol Overhead Analysis:* We analyze the communication and computation overhead in three settings: a failure-free steady state, an average case with occasional view changes, and a worst case with frequent primary failures.

Failure-free steady state. Under stable network conditions without faults, each consensus round relies on the threshold-signature mechanism. The message volume grows linearly with the number of nodes n , and no global all-to-all broadcast is required. The verification cost per proof is $O(1)$, which is lower than that of aggregate-signature schemes that typically require $O(n)$ verification per round.

Average case. Assume that a single view change occurs. Because the new primary collects $2f+1$ *viewchange* messages and then issues a *newview* message, the per-round communication cost remains linear. The threshold-signature verification cost also remains $O(1)$.

Worst case. Each view change incurs $O(n)$ communication, so v_{max} view changes result in $O(nv_{max})$ messages. In the asymptotic worst case, where $v_{max} = O(n)$, this becomes $O(n^2)$. In our protocol, v_{max} is bounded by a small constant per epoch, so the practical per-epoch overhead remains linear in n .

Practical optimization. Although the asymptotic worst-case communication complexity is unchanged, our design benefits from threshold-signature compression and synchronized view-change optimization. By performing *HighQC* chain-consistency verification to ensure that all replicas agree on the same highest QC, a new primary during a view change needs to disseminate only a single threshold-signature proof of that highest QC, without re-aggregating large QCs. This approach reduces redundant messages and lowers the effective number of communication rounds in practice, thereby improving overall communication efficiency.

2) *Cross-Pool Coordination Overhead Analysis:* Let n denote the number of replicas in each pool-level SMR instance, M denote the number of cross-pool records admitted into a GBP window, and M_j denote the number of records projected to destination pool $Pool_j$. The cross-pool path contains three stages.

In the source-pool commitment stage. Transactions are committed through the source pool's consensus, whose communication cost is $O(n)$ per block and whose threshold-signature verification cost is $O(1)$. After commitment, only certified cross-pool output records are extracted and submitted to the GBP.

In the GBP-pool ordering stage. The GBP acts as a dedicated pool-level SMR instance. It verifies source-pool evidence,

checks replay status, orders admitted records, and commits them in its own log. The GBP-pool ordering stage has the same asymptotic communication cost as one ordinary pool-level consensus instance and does not introduce a separate global all-to-all consensus layer. The communication cost of a GBP block is $O(n)$, and the computation cost is $O(M)$ for verification and replay checks, plus $O(M \log M)$ when deterministic sorting is required. A higher cross-pool ratio increases the number of records processed by the GBP in each window. However, these records are verified and ordered in batches, and regular pools can continue committing local blocks in parallel. Thus, GBP increases the cross-pool coordination workload, but it does not turn Akaverse into a single serial execution pipeline. This design trades a batchable GBP-side ordering step for simpler destination-pool materialization. Relative to direct verification, the GBP layer renders unnecessary the independent collection of source-pool evidence, the maintenance of per-source synchronization state, the reconciliation of out-of-order deliveries, and the establishment of deterministic cross-source ordering at each destination pool. It consolidates these coordination overheads into a unified, batchable sequencing stage. Destination-pool consensus remains autonomous. Source-QC validation stays intact.

In the destination pool materialization stage. Each destination pool verifies its projected records and commits the corresponding target-side events through its own SMR log. For destination pool $Pool_j$, the verification cost is $O(M_j)$ and the consensus communication cost is $O(n)$ per committed target block. Since $\sum_j M_j = M$, the total verification cost across destination pools is $O(M)$.

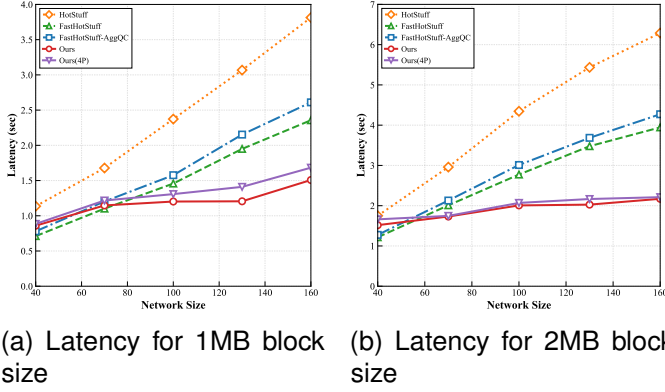
V. PERFORMANCE AND ANALYSIS

We implemented a prototype of the proposed protocol and deployed it on cloud servers to evaluate its practical performance. The Akaverse prototype system was implemented in C++ on CentOS 7.9. The implementation includes the consensus protocol, buffer mechanism, and random number generation process, with all nodes deployed across different geographic regions.

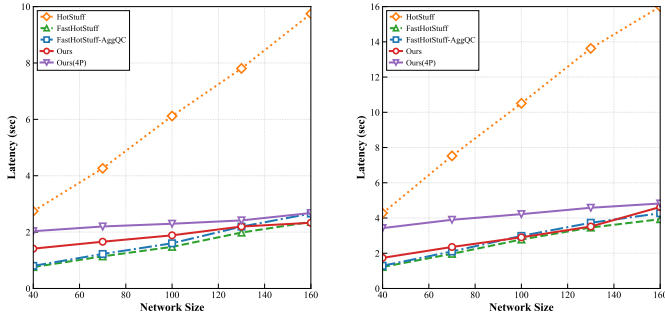
A. Throughput and Latency Analysis

We evaluate Akaverse and Akaverse-4P (four parallel transaction pools) against pipelined Fast-HotStuff, Fast-HotStuff-AggQC, and HotStuff. The network size varies over (40, 70, 100, 130, 160) replicas and the block size over (1 MB, 2 MB). Replicas run on identical Alibaba Cloud ECS VMs; each VM has 4 vCPUs and hosts four replica processes pinned to one vCPU each, with the NIC capped at 500 Mbps. In the LAN setting, all replicas are deployed in the same region on a low-loss link. To study wide-area effects, the WAN setting emulates 100 ms latency with 20 ms jitter and 0.02% packet loss. For the forking-attack experiments, we inject an attack with probability 0.3 per consensus round. For each network size, we run 1,000 attacked rounds and report throughput and the confirmation latency of blocks affected by the attack.

Under the happy-path case, as shown in Fig. 4(a)(b) and Fig. 5(a)(b), Akaverse and Akaverse-4P maintain the lowest



(a) Latency for 1MB block size (b) Latency for 2MB block size



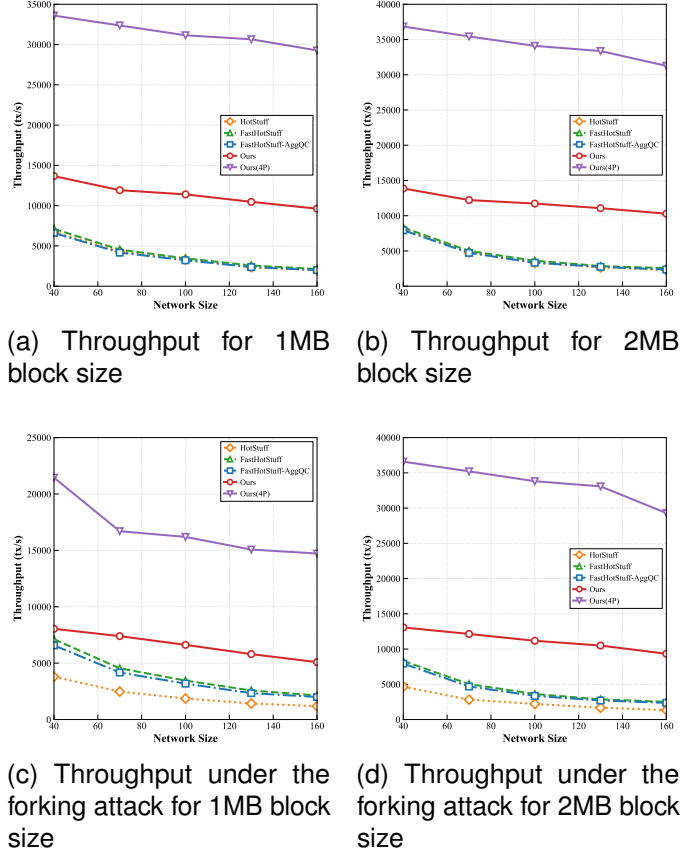
(c) Latency under the forking attack for 1MB block size (d) Latency under the forking attack for 2MB block size

Fig. 4. Latency comparison in LAN environment.

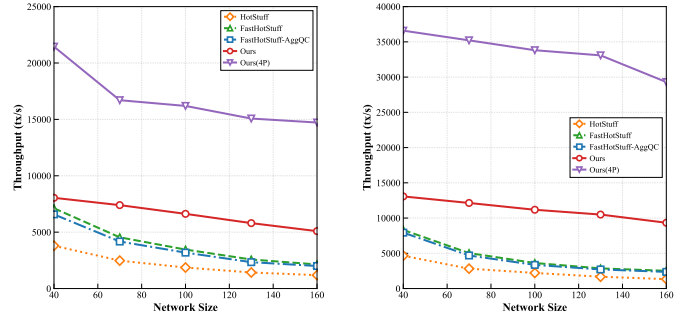
end-to-end latency across all network sizes. As the network grows from 40 to 160 replicas, the latency of HotStuff and both Fast-HotStuff variants increases more sharply, whereas the increase for Akaverse and Akaverse-4P remains modest. With a 1 MB block at 160 replicas, Akaverse-4P confirms blocks in about 1–2 s, while HotStuff requires around 3–4 s, with Fast-HotStuff and Fast-HotStuff-AggQC falling in between. With 2 MB blocks, all curves shift upward, but the relative ordering remains unchanged. Akaverse-4P achieves the highest throughput at every scale and degrades most gradually as the network grows, with Akaverse ranking second. Although throughput decreases overall for 2 MB blocks, Akaverse-4P retains a clear performance advantage.

The gains of Akaverse-4P stem from its parallel transaction pools and pipelined batching path, which shortens the critical round trip. By contrast, HotStuff incurs increasing interpolation overhead at the leader when constructing a t -out-of- n threshold signature, whose cost grows with t . Fast-HotStuff employs a single-pool pipeline and is constrained by per-core execution and network round trips, leading to earlier saturation and lower peak throughput.

As shown in Fig. 4(c)(d) and Fig. 5(c)(d), under the forking attack, the latency of the HotStuff variants increases markedly for both 1 MB and 2 MB blocks. In contrast, Akaverse and Akaverse-4P exhibit only a small increase. Throughput degrades significantly for HotStuff and Fast-HotStuff under attack, whereas Akaverse-4P remains high and stable. For a 1



(a) Throughput for 1MB block size (b) Throughput for 2MB block size



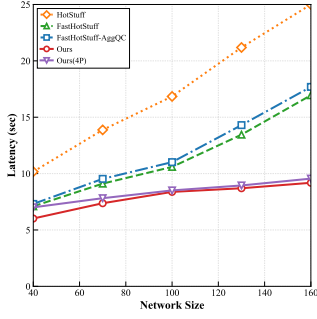
(c) Throughput under the forking attack for 1MB block size (d) Throughput under the forking attack for 2MB block size

Fig. 5. Throughput comparison in LAN environment.

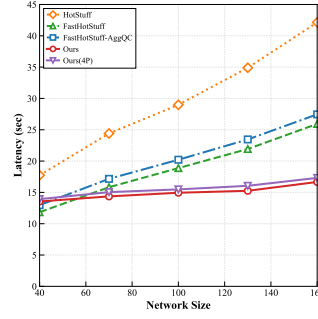
MB block at 160 replicas, Akaverse-4P sustains ≈ 13 – 20 k tx/s, while Fast-HotStuff drops to the low thousands and HotStuff declines even further. Akaverse rejects proposals carrying stale QCs or non-monotonic view numbers and gates block admission through EVS-based view synchronization. This mechanism quickly prunes old-QC fork branches and prevents prolonged commit latencies.

WAN conditions increase end-to-end latency for all protocols relative to LAN. As shown in Fig. 6(a)(b) and Fig. 7(a)(b), Akaverse and Akaverse-4P maintain the lowest latency across all scales, and their latency grows more slowly as the replica count increases. With 1 MB blocks at 160 replicas, HotStuff confirms blocks in roughly 25 seconds, whereas Akaverse and Akaverse-4P remain in the single-digit to low-teens seconds range, making the advantage more pronounced at larger scales. Overall throughput declines in the WAN setting, yet the relative ordering persists: Akaverse-4P remains the highest and Akaverse ranks second. With 1 MB blocks at 160 replicas, Akaverse-4P sustains throughput above 8k tx/s.

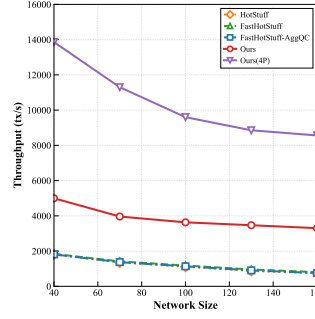
WAN conditions amplify the impact of forking attacks. As shown in Fig. 6(c)(d) and Fig. 7(c)(d), with 1 MB blocks the latency of Fast-HotStuff can climb to ≈ 60 s; with 2 MB blocks, the worst-case latency of HotStuff exceeds 100 s. In contrast, Akaverse and Akaverse-4P show only moderate increases and remain the lowest across scales. Throughput likewise degrades for HotStuff and Fast-HotStuff under attack. Akaverse-4P also declines but remains above all baselines. Notably, its 2 MB



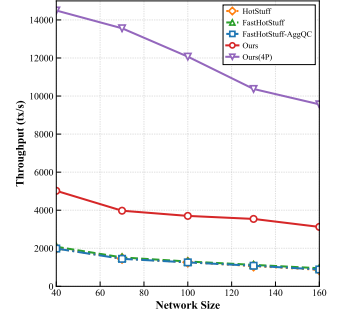
(a) Latency for 1MB block size



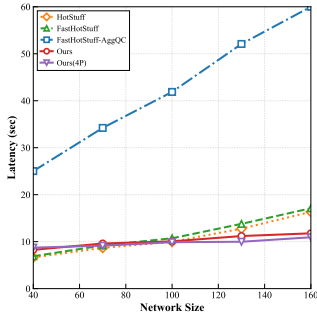
(b) Latency for 2MB block size



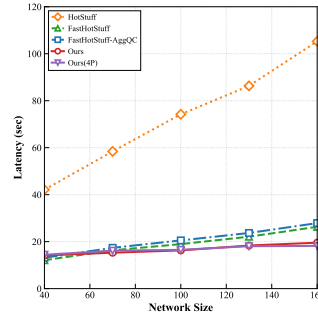
(a) Throughput for 1MB block size



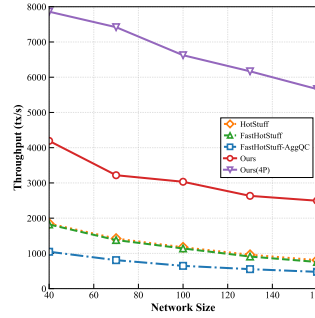
(b) Throughput for 2MB block size



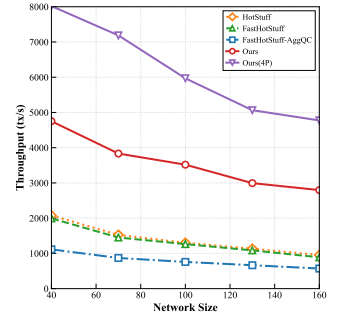
(c) Latency under the forking attack for 1MB block size



(d) Latency under the forking attack for 2MB block size



(c) Throughput under the forking attack for 1MB block size



(d) Throughput under the forking attack for 2MB block size

Fig. 6. Latency comparison in WAN environment.

Fig. 7. Throughput comparison in WAN environment.

throughput drops below its 1 MB value, which we attribute to attack-induced consensus jitter in the WAN; however, the impact remains within a controlled range.

High network latency in WAN environments prolongs the waiting time required by multi-round commit rules in chained protocols. When a fork occurs, HotStuff must re-establish the two-direct-chain condition before committing, which extends confirmation time. Akaverse mitigates this by combining parallel pools with anti-fork admission and strong synchronization. Within the same view, it quickly rejects stale QC proposals and drives progress until a higher valid view forms, thereby constraining the attack's impact on both latency and throughput.

B. Comparison with Randomness Generation

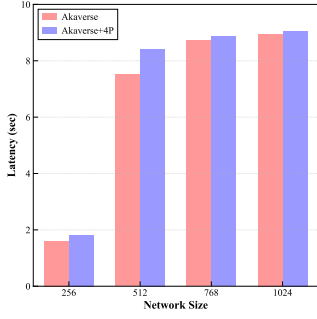
For the randomness-generation overhead comparison, we conduct experiments on a local machine equipped with an AMD Ryzen 7950X CPU and 64 GB of memory. We evaluate the efficiency of Akaverse in secure randomness generation by comparing it with the widely adopted verifiable randomness protocol RandHound. The experiments cover network sizes $N = 10, 30, 50, 100, 150, 200, 300, 500$, measuring latency, message count, and bandwidth consumption. As illustrated in Fig. 10, Akaverse consistently outperforms RandHound across all tested scales. For $N = 200$, latency decreases from 28.403 s to 4.0836 s, representing a $7\times$ performance improvement, while bandwidth usage drops from 173.1 KB to 33.3 KB. These

results demonstrate that the randomness-generation approach in Akaverse is not only lightweight and efficient but also capable of maintaining scalability in large-scale distributed consensus systems, making it a practical and reliable alternative.

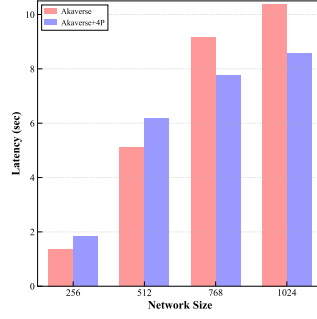
C. Large-Scale Performance

We stress-test Akaverse and Akaverse-4P at large scales of 256, 512, 768, and 1024 replicas. Each Alibaba Cloud server has 4 vCPUs and co-locates four replica processes (one vCPU per replica). The bandwidth is capped at 500 Mbps. In the large-scale LAN setting, we fix the network latency at 50 ms and evaluate 1 MB and 2 MB blocks, reporting throughput and latency. In the large-scale WAN setting, we emulate 100 ms latency with 20 ms jitter and 0.01% packet loss and evaluate 300 KB and 600 KB blocks. **All transactions are cross-pool.** All other software configurations are identical across conditions.

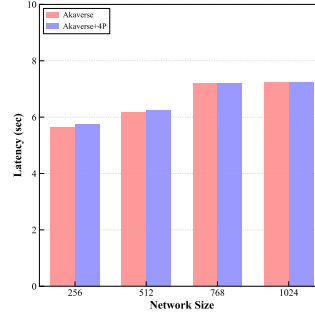
In large-scale LAN environments, as the system scales from 256 to 1024 replicas, both latency and throughput remain within practical ranges. Fig. 8 shows that with 1 MB and 2 MB blocks, Akaverse and Akaverse-4P maintain confirmation latency of roughly 9–10 s, demonstrating stable performance at scales of up to 1024 replicas. Throughput highlights the scalability of multi-pool parallelism: Akaverse-4P exceeds 32k tx/s at 256 replicas and sustains 18–19k tx/s at 1024 replicas, delivering a substantial gain over the single-pool variant while keeping latency growth modest.



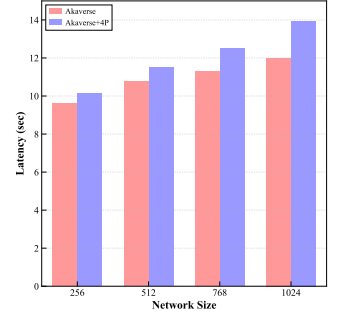
(a) Latency for 1MB block size



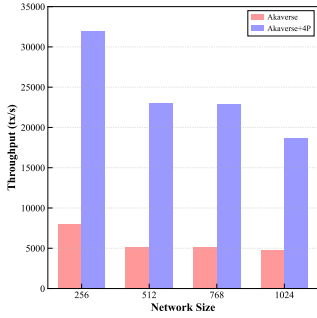
(b) Latency for 2MB block size



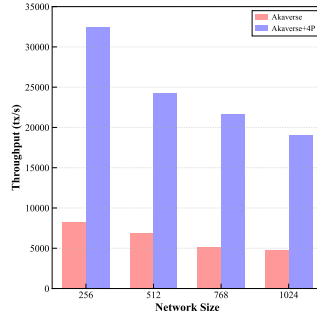
(a) Latency for 300KB block size



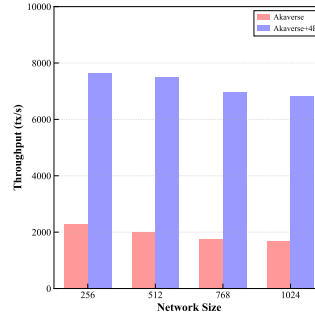
(b) Latency for 600KB block size



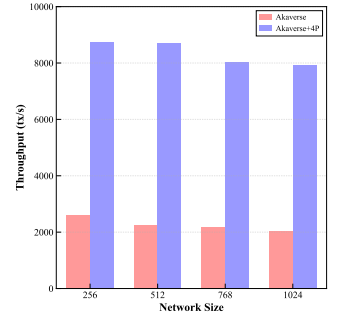
(c) Throughput for 1MB block size



(d) Throughput for 2MB block size



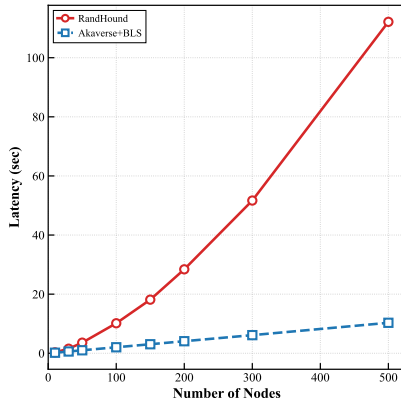
(c) Throughput for 300KB block size



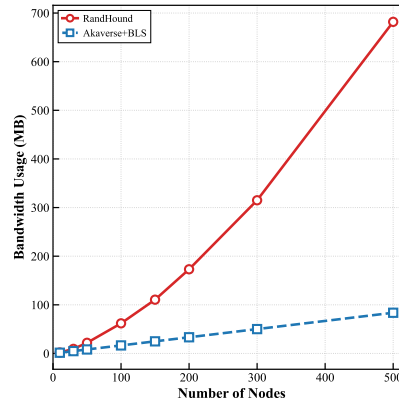
(d) Throughput for 600KB block size

Fig. 8. Performance in large-scale LAN environment.

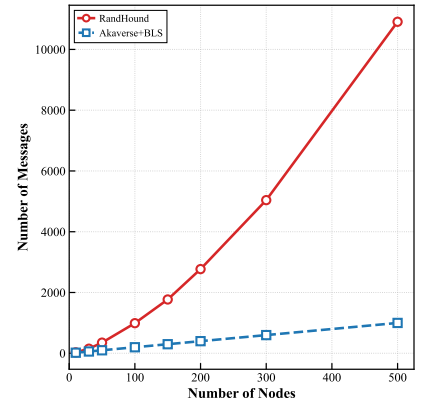
Fig. 9. Performance in large-scale WAN environment.



(a) Latency



(b) Bandwidth



(c) Number of messages

Fig. 10. Performance evaluation of randomness generation.

In large-scale WAN environments, latency increases as expected and exhibits higher variance due to jitter, yet the overall trend remains stable. As shown in Fig. 9, with 300 KB blocks latency is 5-6 s at 256 replicas and increases moderately up to 1024 replicas; with 600 KB blocks, it ranges from 10 to 14 s. In the multi-pool setting, throughput remains between 6.5 and 8K tx/s, yielding a 3-4× advantage over the single-pool design.

These results show that the multi-pool architecture of Akaverse scales to 1024 replicas while preserving practical

confirmation latency and significantly higher throughput in both LAN and WAN environments. By combining parallel transaction pools with the GBP coordination path, Akaverse-4P shortens the critical path and avoids single-pool saturation, making it well-suited for large-scale distributed deployments.

D. Stress Testing of the GBP

To examine the worst-case behavior of the GBP, we conduct a stress evaluation under fully cross-pool workloads where

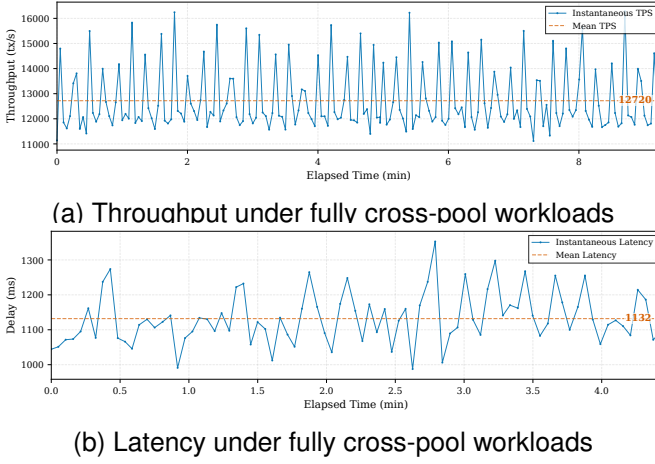


Fig. 11. Throughput and latency under fully cross-pool workloads.

all transactions enter the cross-pool processing path. The experiment uses 256 nodes and 32 transaction pools. Each node is deployed on an Alibaba Cloud server with 8 vCPUs and 16 GB memory. The network is configured with 50 ms latency, 10 ms jitter, and 0.01% packet loss. The maximum block size is set to 1 MB. The fully cross-pool setting forces all transactions to traverse the cross-pool processing path, thereby representing a worst-case stress scenario for GBP.

As shown in Fig. 11, Akaverse achieves an average throughput of 12.7k tx/s under this setting, with an average latency of 1.132 s. These results show that GBP does not cause throughput collapse or persistent latency accumulation even under fully cross-pool workloads.

To study the impact of GBP window size and block size on performance, we vary block size across 100-500 KB (step:100 KB) and δ across 5-20 s (step: 5 s). System throughput and confirmation latency are logged per configuration. Results are plotted in Fig. 12. First, throughput results show a clear upward trend with block size. Larger blocks boost overall throughput. Under a fixed GBP window, each batch absorbs more transactions as block size grows. The fixed costs of consensus and batching then amortize over a wider base. At $\delta = 5$, scaling block size from 100 KB to 500 KB pushes throughput from 17.8k to 21.3k tx/s. The extra capacity translates directly into higher processing power. Second, the δ heavily impacts the latency. Expanding δ from 5 s to 20 s drives latency sharply upward. Transactions sit longer in the buffer pool. This extended wait compounds the overhead of downstream batching, queuing, and consensus, but no meaningful throughput gain in return. The opposite holds for throughput. Across most configurations, a wider window actually drags the rate down. Expanding the time window buys no real performance advantage under the current experimental workload.

We next isolate the waiting overhead under random transaction stress tests. Under the assumption of uniformly distributed transaction arrivals, we estimate the average latency induced by the GBP window. A transaction entering a window of length δ waits roughly $\delta/2$ on average. Fig. 14 plots this theoretical wait as a fraction of the measured latency. At $\delta = 5$ s, the

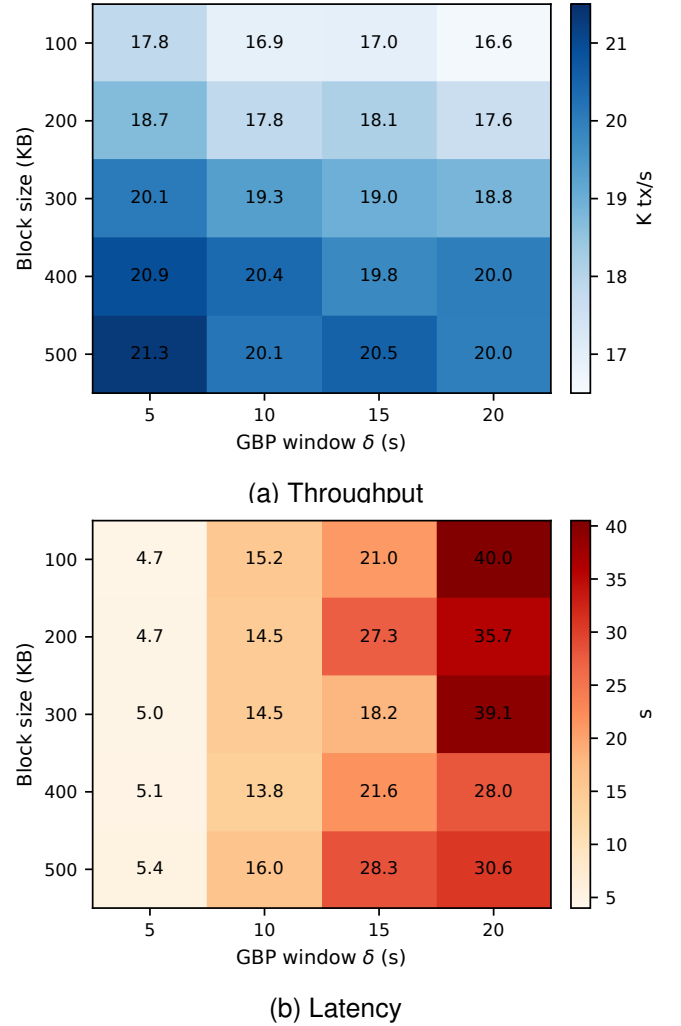


Fig. 12. Impact of GBP window and block size under fully cross-pool workloads.

GBP window accounts for 46%-54% of the total latency. This proportion drops as $\delta = 5$ s grows. It simultaneously inflates the absolute overhead of downstream batching and queuing. These results expose the penalty of oversized windows on latency. A compact δ of 5 s, coupled with a 400 KB or 500 KB block, offers the best throughput-latency tradeoff.

E. Performance under AMM Contract Workloads

To further evaluate Akaverse under realistic smart contract application scenarios, we construct an AMM contract workload that emulates a stateful decentralized exchange environment. Each AMM instance consists of two token contracts and one liquidity-pool contract. This workload follows the co-located AMM deployment mode. Specifically, the token contracts and the liquidity-pool contract of the same AMM instance are assigned to the same transaction pool, which preserves synchronous composability for tightly coupled AMM operations. The workload then executes the core decentralized exchange operations, including liquidity provisioning, approval, and token swaps. Under this deployment, a failed slippage check is handled by the ordinary contract abort semantics

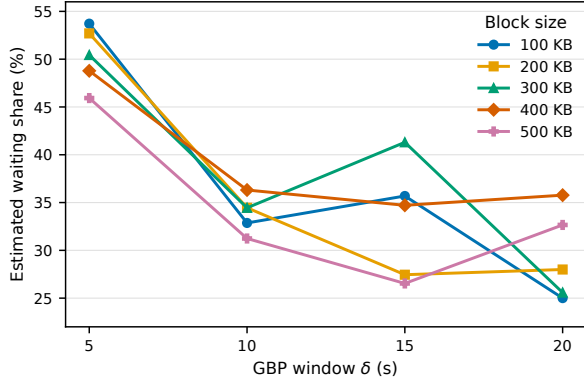


Fig. 13. Estimated GBP-window waiting share under different δ and block sizes.

TABLE II
PER-STAGE EXECUTION RESULTS OF THE AMM CONTRACT WORKLOAD

Stage	Successful Operations	Time Cost
Contract Deployment	3,072	93 s
Transaction Injection	1,024	23 s
Gas Prepayment	150,000	26 s
Token Transfer	50,000	4 s
Token Approval	1,000,000	66 s
AMM Token Swap	1,000,000	66 s

before local commitment, rather than by an asynchronous compensation transaction. Unlike ordinary transfer workloads, AMM execution introduces intensive shared-state updates across multiple interdependent contracts.

We initialize 50,000 user accounts and 1,024 deployer accounts, and deploy 1,024 AMM contract instances. The detailed execution results are reported in Tab. II. These results indicate that Akaverse can sustain high processing capacity even under complex stateful contract workloads. Notably, in the AMM token swap stage, the system completes 1,000,000 swap transactions in 66 s, corresponding to a throughput of more than 1k tx/s.

To further evaluate runtime stability under complex AMM workloads, we separately measure the throughput and latency of approval and swap representative AMM operations. Approval is dominated by allowance-state updates, whereas swap involves more complex cross-contract state transitions. As shown in Fig. ??, the approval workload achieves an average throughput of over 13k tx/s. Despite the more complex state access and write paths in swap execution, Akaverse still sustains 8,774 tx/s for the swap workload. In addition, the average confirmation latencies of approval and swap are 0.769 s and 0.714 s, both remaining below one second. These results indicate that Akaverse avoids persistent latency degradation and sustains high throughput under highly concurrent AMM workloads. These results show that Akaverse sustains efficient execution under highly concurrent stateful AMM workloads.

VI. DISCUSSION

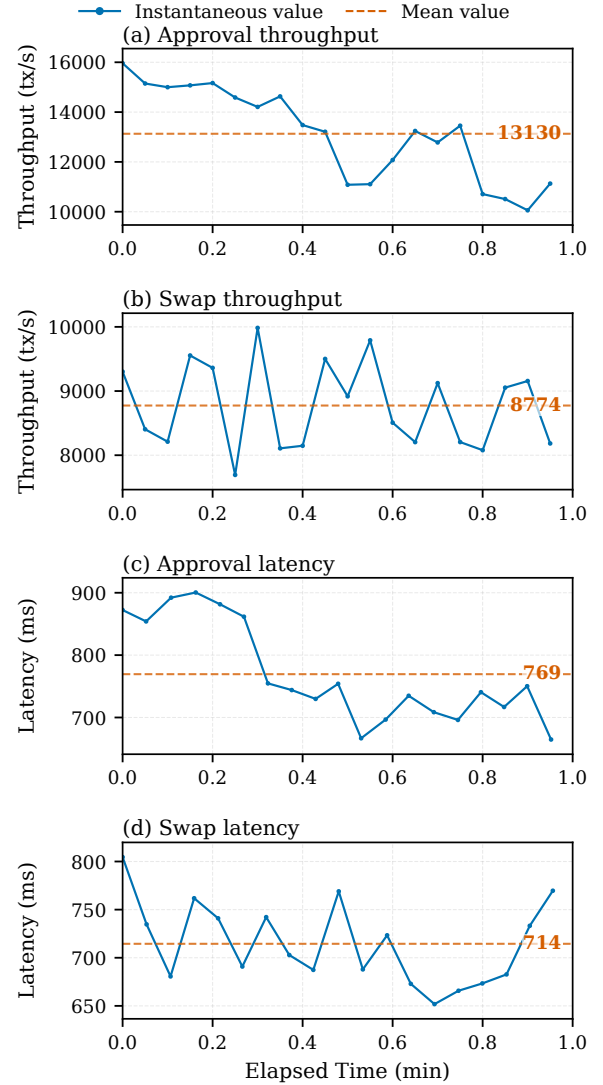


Fig. 14. Throughput and latency of representative AMM operations.

A. Fast-Path Safety Boundary

Akaverse uses different commit rules for normal and recovered executions. Under failure-free consecutive view progression, the protocol may use the restricted two-chain fast rule. Under the recovery-based or non-consecutive view progression, the local flag *Recovered_i* disables the fast path, and replicas use the standard chained HotStuff three-chain commit rule. This design follows directly from our safety analysis. Same-view fork prevention rules out conflicting prepare QCs within a single view, while EVS prevents stale forks from obtaining new prepare QCs after recovery. As a result, Akaverse does not rely on the AggQC of Fast-HotStuff two-chain recovery proof, but instead confines two-chain commits to the normal path and uses the three-chain rule as the recovery safety baseline.

B. GBP Design Rationale

A destination pool could verify source-pool certificates directly, similar to a light-client relay. However, direct verification

would push the full coordination burden to every destination pool. Because records from different sources may arrive in different orders, destination replicas still need a deterministic ordering rule before local materialization.

GBP addresses this issue by introducing a batchable pool-level ordering path for certified cross-pool records. Rather than replacing destination-pool consensus, GBP records a deterministic order once in its own log and exposes ordered projections to destination pools. Final materialization remains under destination-pool SMR.

A direct-verification design removes the GBP ordering step, but each destination pool must independently collect source proofs, maintain per-source progress, handle out-of-order arrivals, and establish its own deterministic order before local commitment. In contrast, GBP performs these coordination tasks once in the GBP log and reuses the resulting ordered projections across destination pools.

Although GBP may appear similar to relay or side-chain coordination systems, its role is different from inter-chain relay architectures such as Cosmos or Polkadot. Those systems connect independent chains or zones through relay protocols, cross-chain light clients, or shared relay-chain security. In contrast, GBP does not introduce an external relay chain, a separate validator set, or an independent inter-chain finality layer.

C. Deployment Semantics for Cross-Pool Contracts

Akaverse reduces the procedural burden of cross-pool coordination, yet the programming model remains sensitive to deployment topology. Co-located entities preserve single-pool atomicity and synchronous composability, making co-location preferable for tightly coupled contracts. Interactions across different pools follow the step-wise execution model of Lemma A7 rather than a single globally atomic transaction. Functionally independent contracts can therefore be distributed across pools to exploit parallelism.

Co-location does not collapse Akaverse into a single-pool design. It is applied only to tightly coupled contract groups whose operations require synchronous composability. Independent AMM instances, independent contracts, and separable workflows remain distributed across pools and continue to exploit parallel execution. Thus, Akaverse treats contract placement as a topology-aware deployment decision: strong state dependencies favor co-location, while weakly coupled workloads favor cross-pool execution.

For AMM-style smart contracts, this distinction leads to two deployment patterns. For latency-critical swaps, Akaverse recommends co-locating the AMM pool and its associated token contracts in the same transaction pool. In this pattern, allowance checks, reserve updates, slippage checks, and token transfers are executed within one pool-level SMR log. If the slippage predicate fails, the swap aborts before local commitment, and no cross-pool compensation is required.

For intentionally cross-pool AMM routing, Akaverse treats the route as a sequence of locally committed steps rather than a single synchronous transaction. Each hop performs its own application-level predicate checks, such as a per-hop minimum-output constraint. If a later hop fails before local commitment,

it produces no ledger-visible effect; already committed assets remain in the last successful state and can be retried, rerouted, refunded, or settled according to the application policy. This design avoids global rollback and makes failure handling explicit and auditable, but it is intended for loosely coupled or multi-hop workflows rather than latency-critical synchronous swaps.

Akaverse follows a sharded SMR semantics rather than a uniform global-log SMR semantics. Regular pools and the GBP pool each maintain their own committed logs. Cross-pool execution proceeds through a certified forward path from the source-pool log to the GBP-pool log, and finally to the destination-pool log. This design preserves source-side causal order for dependent cross-pool records while avoiding the cost of imposing a single global total order on all independent transactions.

Building on Akaverse, we plan to further explore a cross-shard smart-contract operating system. The goal is to move topology-aware deployment and cross-pool orchestration into the runtime, reducing the need for developers to manage deployment topology explicitly.

VII. CONCLUSION

In this paper, we presented Akaverse, a scalable sharded blockchain that combines Fast-HotStuff with BLS threshold signatures and view-synchronization gating under partial synchrony, delivering linear communication and faster view synchronization. We embed a randomness beacon directly into the consensus protocol to provide low-cost, verifiable randomness without relying on external modules. To improve cross-shard efficiency, we propose a multi-leader parallel execution scheme with a dedicated GBP pool, enabling rollback-free cross-pool coordination through source-pool finality, GBP-pool ordering, and destination-pool materialization. Our implementation on 1024 replicas achieves up to 19K tx/s with confirmation latency under 10 s and remains robust even under forking attacks. These results demonstrate that Akaverse provides a practical design for high-throughput and resilient large-scale sharded blockchain protocols with strong consistency guarantees.

ACKNOWLEDGMENTS

The work is supported by the National Key R&D Program of China (No. 2023YFB2703700), the National Natural Science Foundation of China (Nos. 62302457, 62441228), the Fundamental Research Funds of Zhejiang Sci-Tech University under Grants No. 22222266-Y, the Program for Leading Innovative Research Team of Zhejiang Province (No. 2023R01001), the Zhejiang Provincial Natural Science Foundation of China (No. LQ24F020008), the “Pioneer” and “Leading Goose” R&D Program of Zhejiang (Nos. 2025C02033, 2023C01119), the Program of Zhejiang Key Laboratory of Digital Fashion and Data Governance (No. 2024E10049).

REFERENCES

- [1] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, “A secure sharding protocol for open blockchains,” in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 17–30.

- [2] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, "Bigtable: A distributed storage system for structured data," *ACM Transactions on Computer Systems (TOCS)*, vol. 26, no. 2, pp. 1–26, 2008.
- [3] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "Omiledger: A secure, scale-out, decentralized ledger via sharding," in *2018 IEEE symposium on security and privacy (SP)*, 2018, pp. 583–598.
- [4] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 931–948.
- [5] K. Zhou, X. Zhang, C. Wang, and H. Cheng, "Accelerating cross-shard blockchain consensus via decentralized coordinators service with verifiable global states," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1340–1353, 2024.
- [6] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, "Chainspace: A sharded smart contracts platform," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.
- [7] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 95–112.
- [8] Y. Liu, A. Liu, Y. Lu, Z. Pan, Y. Li, J. Liu, S. Bian, and M. Conti, "Kronos: A secure and generic sharding blockchain consensus with optimized overhead," in *Network and Distributed System Security Symposium (NDSS)*, 2025.
- [9] F. Cheng, J. Xiao, C. Liu, S. Zhang, Y. Zhou, B. Li, B. Li, and H. Jin, "Shardag: Scaling dag-based blockchains via adaptive sharding," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 2068–2081.
- [10] Y. Liu, X. Xing, H. Cheng, D. Li, Z. Guan, J. Liu, and Q. Wu, "A flexible sharding blockchain protocol based on cross-shard byzantine fault tolerance," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2276–2291, 2023.
- [11] A. Liu, Y. Liu, Q. Wu, B. Zhao, D. Li, Y. Lu, R. Lu, and W. Susilo, "Cherubim: A secure and highly parallel cross-shard consensus using quadruple pipelined two-phase commit for sharding blockchains," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 3178–3193, 2024.
- [12] A. Singh, T. Das, P. Maniatis, P. Druschel, and T. Roscoe, "BFT protocols under fire," in *Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2008, pp. 189–204.
- [13] A. Clement, E. Wong, L. Alvisi, M. Dahlin, and M. Marchetti, "Making byzantine fault tolerant systems tolerate byzantine faults," in *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2009, pp. 153–168.
- [14] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "Hotstuff: Bft consensus with linearity and responsiveness," in *Proceedings of the 2019 ACM symposium on principles of distributed computing*, 2019, pp. 347–356.
- [15] J. Niu, F. Gai, M. M. Jalalzai, and C. Feng, "On the performance of pipelined hotstuff," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*, 2021, pp. 1–10.
- [16] G. Zhang, F. Pan, Y. Mao, S. Tjanic, M. Dang'ana, S. Motepalli, S. Zhang, and H.-A. Jacobsen, "Reaching consensus in the byzantine empire: A comprehensive review of bft consensus algorithms," *ACM Computing Surveys*, vol. 56, no. 5, Jan. 2024.
- [17] M. M. Jalalzai, J. Niu, C. Feng, and F. Gai, "Fast-hotstuff: A fast and robust bft protocol for blockchains," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 2478–2493, 2023.
- [18] A. Bessani, J. Sousa, and E. E. Alchieri, "State machine replication for the masses with bft-smart," in *2014 44th Annual IEEE/IFIP international conference on dependable systems and networks*, 2014, pp. 355–362.
- [19] M. Bravo, G. Chockler, and A. Gotsman, "Making byzantine consensus live," *Distributed Computing*, vol. 35, no. 6, pp. 503–532, 2022.
- [20] R. Gelashvili, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman, and Z. Xiang, "Jolteon and ditto: Network-adaptive efficient consensus with asynchronous fallback," in *International conference on financial cryptography and data security*, 2022, pp. 296–315.
- [21] L. Yang, S. J. Park, M. Alizadeh, S. Kannan, and D. Tse, "[DispersedLedger]:{High-Throughput} byzantine consensus on variable bandwidth networks," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 493–512.
- [22] A. Spiegelman, N. Girdharan, A. Sonnino, and L. Kokoris-Kogias, "Bullshark: Dag bft protocols made practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2705–2718.
- [23] G. G. Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, "Sbft: A scalable and decentralized trust infrastructure," in *2019 49th Annual IEEE/IFIP international conference on dependable systems and networks (DSN)*, 2019, pp. 568–580.
- [24] X. Meng, X. Sui, Z. Yang, K. Rong, W. Xu, S. Chen, Y. Yan, and S. Duan, "Rondo: Scalable and reconfiguration-friendly randomness beacon," in *Network and Distributed System Security Symposium (NDSS)*, 2025.
- [25] B. David, P. Gazi, A. Kiayias, and A. Russell, "Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2018, pp. 66–98.
- [26] A. Bhat, N. Shrestha, A. Kate, and K. Nayak, "Optrand: Optimistically responsive reconfigurable distributed randomness," in *Network and Distributed System Security Symposium (NDSS)*, 2023.
- [27] A. Bhat, N. Shrestha, Z. Luo, A. Kate, and K. Nayak, "Randpipe: reconfiguration-friendly random beacons with quadratic communication," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 3502–3524.
- [28] E. Syta, P. Jovanovic, E. K. Kogias, N. Gailly, L. Gasser, I. Koffi, M. J. Fischer, and B. Ford, "Scalable bias-resistant distributed randomness," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 444–460.
- [29] B. Schoenmakers, "A simple publicly verifiable secret sharing scheme and its application to electronic voting," in *Annual International Cryptology Conference*, 1999, pp. 148–164.
- [30] D. Beaver, K. Chalkias, M. Kelkar, L. Kokoris-Kogias, K. Lewi, L. de Naurois, V. Nikolaenko, A. Roy, and A. Sonnino, "STROBE: Streaming threshold random beacons," in *5th Conference on Advances in Financial Technologies (AFT 2023)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 282, 2023, pp. 7:1–7:16.